

Internship Portfolio

Exploring Emergent Modularity in CNNs of Varying Capacity via Systematic Ablation

Student

Name: Chiara Emma Wermter

SNR: 2040429

ANR: 155827

MSc Cognitive Science and Artificial Intelligence

Internship Provider

Tilburg University

Supervisor: Dr. Ifigeneia Mavridou

University Supervisors

Dr. Ifigeneia Mavridou

Dr. Grzegorz Chrupała

Contents

1 Introduction **3**
- 1.1 Related Work 4
  - 1.1.1 Sleep Staging 4
  - 1.1.2 Modularity: Designed & Emergent 4
- 1.2 Analysis of Csordás et al.'s Methodology 5
  - 1.2.1 Overview of Differentiable Weight Masking Approach 5
  - 1.2.2 Mask Training across Architectures 6

2 Methods **6**
- 2.1 Data and Pre-processing 6
- 2.2 Pre-processing Pipelines for Input Disentanglement 6
  - 2.2.1 Data Splits 7
- 2.3 Classification Models 7
  - 2.3.1 Baseline models 7
  - 2.3.2 Models for Experimentation 7
- 2.4 Differentiable Weight Masking . 7
  - 2.4.1 Mask Training Procedure 8
  - 2.4.2 Loss Function and Regularization 8
  - 2.4.3 Methodological Adaptations 8
- 2.5 Operational Definitions for Network Analysis 8
  - 2.5.1 Excluded weights of a class c 8
  - 2.5.2 Core Weights 8
  - 2.5.3 Reconstructed Class Weights 9
  - 2.5.4 Shared weights between classes c and d 9
- 2.6 Derived Network Component Metrics 9
  - 2.6.1 Exclusion Proportion . . 9
  - 2.6.2 Global Exclusion 9
  - 2.6.3 Pairwise Sharing Proportion 9
  - 2.6.4 Global Pairwise Sharing 9
  - 2.6.5 Summary 9

3 Results **10**
- 3.1 Classification Models 10
  - 3.1.1 Baseline Models 10
  - 3.1.2 Selected Models 10
- 3.2 Differentiable Weight Masking . 11
  - 3.2.1 Modularity Across Conditions 11
  - 3.2.2 Core Network 12
  - 3.2.3 Effect of Exclusion and Reconstruction on Performance 13

4 Discussion **14**
- 4.1 Limitations and Methodological Considerations 15
  - 4.1.1 Class-Exclusion Mask Training 15
  - 4.1.2 Scope and Generalizability 16
  - 4.1.3 Statistical Power 16
- 4.2 Implications and Future Research 16
  - 4.2.1 Methodological Extensions 16
  - 4.2.2 Architectural Research . 16
  - 4.2.3 Signal Processing Research 16
- 4.3 Conclusion 17

A Study Appendix **20**
- A.1 Conceptualizations of Emergent Modularity in Literature 20
- A.2 Detailed Data and Preprocessing Specifications 20
- A.3 Detailed Model Architectures . 22
- A.4 Detailed Loss Function and Regularization 23
- A.5 Detailed Methodological Adaptations for Mask Training . . . 23
- A.6 Full Network: Performance and Modularity Metrics 23
- A.7 Core Network: Performance . . 24
  - A.7.1 Summary Statistics . . . 24
  - A.7.2 Accuracy and F1 in Detail 24
- A.8 Reconstruction and Exclusion: Performance changes 25
  - A.8.1 Summary Statistics . . . 25
  - A.8.2 Corresponding Class . . 25
  - A.8.3 Across Classes 27
  - A.8.4 Confusion Matrices: Class-Exclusion 30
  - A.8.5 Confusion Matrices: Class-Inclusion 33
  - A.8.3 Confusion Matrices: Class-Exclusion 36
  - A.8.4 Confusion Matrices: Class-Inclusion 36

B Personal Development Section **36**

B.1 Personal Evaluation of the Internship 36
B.2 Evaluation of Personal Learning Objectives 37

C Statement of Technology 38

D Portfolio Appendix 39
D.1 Assessment survey 39
D.2 Proposal 41

E Activity Log 52

1 Introduction

Neural networks achieve high performance on complex classification tasks (Saganowski et al., 2019), but often lack interpretability. Networks trained on noisy and subject-specific physiological data especially suffer from performance degradation on out-of-distribution data (Li et al., 2025; Y. Liu et al., 2025; Pfeiffer et al., 2023). Data scarcity, high inter-subject variability, and expensive labeling in the medical domain make scaling an unsustainable solution, amplifying generalisation issues (Y. Liu et al., 2025).

Biological neural systems achieve generalisation by decomposing complex tasks into recurring components (Lorenz et al., 2011). The components are reused for different purposes without losing their original functionality (Anderson, 2010). This reuse could enable a neural network to utilize training data more efficiently. Indeed, many NNs decompose complex tasks into modular subroutines (Lepori et al., 2023). The formed connectivity patterns are similar to those of biological modules (Csordás et al., 2020; Filan et al., 2021), where modules are densely interconnected with sparse connections between them (Alcalá-Corona et al., 2021).

However, Csordás et al. (2020) concluded that typical NNs do not tend to exhibit the desired reuse. Sharing between subnetworks did not increase with task similarity but with lower capacity. It is unclear whether their observation holds when trained on different inputs, as Béna and Goodman (2025) found specialization is more likely to emerge when input features are statistically independent.
Thus, several research gaps remain. Most investigations use simplified problems rather than realistic domains like physiological signal classification, where modularity could offer practical benefits. While capacity constraints appear to influence modularity, no study has systematically examined how network size interacts with pre-processing.
Lastly, despite Csordás et al. (2020) concluding that CNNs fail to share weights, their sharing metric was never calculated for CNNs. Instead, they based their conclusion on a sharp drop in performance for individual classes when removing class-specific weights. However, a performance drop following removal does not demonstrate that the weights which remain are non-functional. A network could simultaneously contain (a) class-specialized weights whose removal harms performance, and (b) a shared set of weights capable of meaningful classification on its own.
This study addresses these gaps by investigating emergent modularity in 1D CNNs trained on physiological sleep staging data. Network capacity and input disentanglement are systematically varied across three pre-processing pipelines to test whether the class-specific and shared weights are functionally competent in CNNs. To assess whether shared weights show functionality, a core network is introduced consisting of the set of weights that remain when excluding all class-excluded weights. This isolated network is tested independently. Reuse, as conceptualized by Csordás et al. (2020), will not be tested in the study. In addition, it is tested whether adding these class-specific weights to the core network selectively restores performance for that class.

This investigation is guided by several research questions. Before examining modularity, three preliminary questions establish that the experimental setup is viable and that capacity can be varied.

1. **RQ1.1 (Feasibility):** Can a 1D CNN model successfully learn to recognize sleep stages from the pre-processed, multimodal time-series data (EMG, EOG, EEG and ECG)?
2. **RQ1.2 (Input Disentanglement):** What pre-processing pipeline can be used to reduce mutual information between all physiological sensor channels?
3. **RQ1.3 (Capacity Baseline):** What is the smallest network architecture that achieves sufficient performance (>70% accuracy), providing a baseline for systematically varying capacity in the main modularity analysis?

The main investigation is concerned with characteristics of modular organization and what functional roles different weight sets play, thereby addressing some underlying assump-

tions of Csordás et al. (2020).

4. **RQ2.1 (Organizational Structure):** How do network capacity and pre-processing affect (a) the proportion of class-exclusive weights and (b) the degree of weight sharing, showing whether CNNs organize computation in exclusive or shared pathways?

5. **RQ2.2 (Shared Functionality):** Do CNNs develop a functionally competent core network, (a) where weights shared across all classes can classify on their own and (b) how does this core competence vary with capacity and pre-processing?

6. **RQ2.3 (Specialized Function):** Do class-excluded weights function as dedicated class detectors, such that their removal selectively harms target class performance and their addition to the core network selectively restores it?

1.1 Related Work

1.1.1 Sleep Staging

Sleep stage classification was selected because the task contains both stages with distinct features and stages on a continuum, offering a test bed where a network would benefit from both specialized and shared computation. Beyond its experimental utility, sleep staging research enables better diagnostics and insight into neurological processes (Lambert & Peter-Derex, 2023). Sleep is divided into Non-Rapid Eye Movement (NREM) and Rapid Eye Movement (REM) states via Polysomnography (PSG), which integrates EEG, EOG, and EMG signals. Following AASM guidelines, signals are divided into 30-second epochs and categorized as Wake (W), N1, N2, N3, or REM (Malhotra & Avidan, 2013).

Wakefulness shows prominent alpha rhythm (8-13 Hz) and high muscle tone.

Stage N1 is characterized by slow eye movements and mixed-frequency theta activity (4-7 Hz).

Stage N2 is defined by sleep spindles and K-complexes.

Stage N3 (deep sleep) is identified when high-amplitude slow waves (0.5-2 Hz) occupy >20% of the epoch (Lambert & Peter-Derex, 2023). NREM stages are associated with declarative memory stabilization; the glymphatic system is most active during N3, clearing metabolites like beta-amyloid (Woidtke, 2024).

REM sleep shows desynchronized, wake-like EEG, rapid eye movements, and muscle atonia (low chin EMG), and is associated with emotional regulation and procedural memory (Lambert & Peter-Derex, 2023; Woidtke, 2024).

Distinguishing N1, N2, and N3 is particularly challenging. N2 and N3 exist on a slow-wave continuum, and N1 vs. N2 often hinges on a single spindle or K-complex (Lee et al., 2021; Malhotra & Avidan, 2013). In contrast, REM and Wake are more distinct: N1 and REM share similar low-amplitude EEG but differ in EMG/EOG; Wake and N3 differ via non-overlapping alpha and delta activity; Wake and REM share EEG patterns but are highly distinct in EMG tone (Lambert & Peter-Derex, 2023). Thus, sleep staging enables investigation of emergent modularity in realistic physiological signals, as some stages share significant signal characteristics while others have highly divergent profiles.

Having established sleep staging as a suitable domain, the conceptualizations and investigations of modularity in neural networks are reviewed.

1.1.2 Modularity: Designed & Emergent

Modularity in Systems and Neural Networks In technical domains, modular systems are decomposable; their components are relatively independent and can be repurposed in different configurations (Amer & Maul, 2019; Praczyk, 2023). NNs can be designed to have a similar modular structure (Qiu et al., 2023), enabling modules to solve common subcomponents across different tasks (Filan et al., 2021). Such modular networks can achieve higher generalisation, than non-modular baselines on compositional tasks (Ruis & Lake, 2022). However, this explicit construction requires prior knowledge about the tasks (Z. Liu et al., 2023; Ruis & Lake, 2022). Perhaps the complexity of design is why many NNs rely on non-modular systems (Praczyk, 2023).

Emergent Modularity An alternative is to create conditions under which modularity emerges spontaneously. Filan et al. (2021) observed that Multi-Layer Perceptrons develop module-like connectivity when subjected to dropout or pruning. Qiu et al. (2023) demonstrated that modules discovered within a monolithic network can be used to build modular networks that exceed the generalisation abilities of comparable finetuned monolithic networks. However, Qiu et al. (2023) and Filan et al. (2021) used entirely different conceptualizations of the term "module".
Indeed, existing research on emergent modularity follows different paradigms ranging over structural, functional and data representational perspectives. A non-exhaustive overview can be found in Appendix Table 6.

The present study focuses on the functional paradigm, in which modules are defined solely by function with no regard to their structure. Within this paradigm several conditions are known to facilitate module emergence: narrow architectural bottlenecks, statistically independent input features, and multi-task learning objectives (Béna & Goodman, 2025; Johnston & Fusi, 2023; Lepori et al., 2023).

To investigate how input processing and network capacity shape emergent modularity, the differentiable weight masking framework of Csordás et al. (2020) is adopted.

1.2 Analysis of Csordás et al.'s Methodology

1.2.1 Overview of Differentiable Weight Masking Approach

To identify functional modules, Csordás et al. (2020) introduced differentiable weight masking. Their approach (Figure 1): (1) trains the network to convergence, (2) freezes weights, (3) identifies subsets of weights necessary for each subtask/class by training binary masks. The identified weight subsets are considered the functional modules that perform a specific function. To assess how much networks share their weights, the overlap of the modules is quantified. Modules are defined as "subnetworks needed to perform specific function[s]". Networks can (1) use "different modules for very different functions", called P1 or (2) use "the same module for identical functions", called P2. A network with both properties, P1 and P2, supports compositionality (Csordás et al., 2020).

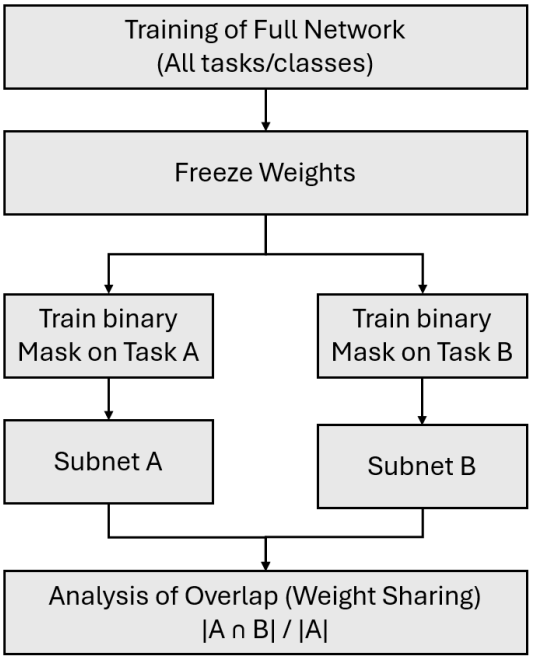


Figure 1: Procedure used for analysis of weight sharing.

The properties are investigated by assessing the overlap and function of modules in several networks. To investigate P1, the network was trained on different tasks, on the assumption that this would force the model to develop separate, specialized computation pathways. In line with expectations, modules shared few weights and failed to perform the complementary task.

To encourage P2, a model was trained on identical tasks presented through different input channels. The expectation, that identified modules would show strong overlap and be functionally interdependent, was violated, leading to the conclusion that common models fail to share weights.

While this approach is innovative, several conceptual, methodological, and analytical issues limit its interpretative power. Capacity effects were examined only for P1 (more sharing in smaller networks). Whether P2 behaves similarly under capacity constraints remains untested.

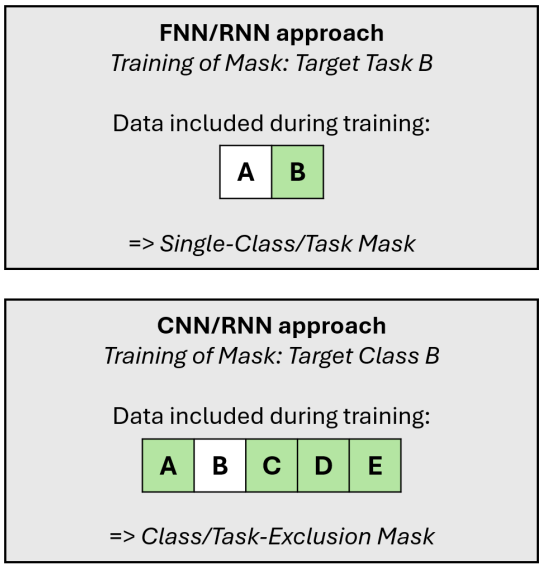


Figure 2: Differences in approaches when identifying target masks

1.2.2 Mask Training across Architectures

More critically, mask training differs across architectures. The approaches used for different investigations deviate strongly. Masks for FNNs and most RNNs are trained using samples of **only one target task** (hereafter termed Single-Class or Single-Task Masks). In contrast, masks for CNNs were trained to classify **all but one target class**, a procedure also used for RNNs trained on more than two tasks. These will be termed Class- or Task-Exclusion Masks. The difference between Single-Task and Task-Exclusion masks is illustrated in Figure 2.

Due to this difference, the introduced sharing metric is not suitable to assess sharing in CNNs. The metric quantifies overlap between two different single-class masks, but CNN masks were trained as class-exclusion masks. Therefore, the training data for masks M_B and M_C necessarily overlap: a mask for class B is trained on samples of A, C, D, E, and a mask for class C on samples of A, B, D, E (as illustrated in Figure 3). Any measured weight overlap could reflect shared training data rather than functional similarity between classes B and C.

While Csordás et al. (2020) concluded that *"RNNs and CNNs exhibit a consistent pattern of failing to share weights"*, their sharing metric was not applied to the CNN. Instead, the proportion of excluded weights and the performance of the class-exclusion mask on the full test data set were assessed. A performance decrease in the excluded class was interpreted as heavy reliance on class-exclusive, non-shared features. However, the necessity of a weight set does not inherently prove the absence of a reused core.

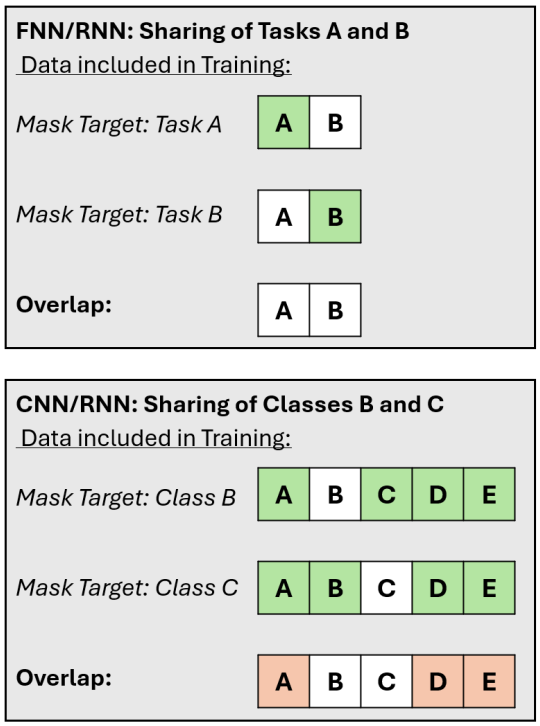


Figure 3: Training data overlap between FNN/RNN masks and CNN masks.

2 Methods

2.1 Data and Pre-processing

The ISRUC-Sleep dataset is used (Khalighi et al., 2015) containing overnight polysomnography recordings from 58 adult subjects (after quality filtering). Each 30-second epoch includes 6 EEG, 2 EOG, 1 EMG, and 1 ECG channel sampled at 200 Hz, annotated with sleep stages (Wake, N1, N2, N3, REM).

2.2 Pre-processing Pipelines for Input Disentanglement

Three pre-processing pipelines are implemented to systematically vary input disentanglement. Therefore, Pipeline 1 provides entangled, high-dimensional inputs; Pipeline 2 offers partially disentangled features; Pipeline 3 yields maximally disentangled, low-dimensional representa-

tions. This progression builds on Johnston and Fusi (2023)'s hypothesis that disentangled inputs promote specialized sub-populations. See Appendix A.2 for details about data and preprocessing.

Pipeline 1 (Raw-filtered): Minimal processing preserving signal entanglement. All channels are notch-filtered at 50Hz with modality-specific bandpass filtering (EEG: 0.3-35Hz, EOG: 0.1-15Hz, EMG: envelope extraction 10-99Hz $\rightarrow$ 4Hz lowpass, ECG: 0.5-45Hz).

Pipeline 2 (Bandpower features): Moderate disentanglement via time-varying bandpower features. Using 10-second sliding windows, standard frequency bands are extracted per modality (e.g., EEG: delta, theta, alpha, sigma, beta).

Pipeline 3 (PCA-reduced): Maximum disentanglement via PCA. Pipeline 2 features are reduced to 20 components explaining 95% variance, whitening for equal variance.

2.2.1 Data Splits

Participant-wise stratified data splits (70% train, 15% validation, 15% test) with 30,833 training, 5,625 validation, and 7,018 test epochs are used. Only epochs with expert agreement are included to ensure label reliability.

2.3 Classification Models

This study uses 1D CNNs to investigate how input representation and network capacity influence emergent modularity. All models share three convolutional layers with decreasing kernel sizes (7, 5, 3) to capture coarse-to-fine temporal patterns, followed by global average pooling to create an information bottleneck that encourages filter specialization (Csordás et al., 2020). ReLU activations, max pooling between layers, and a single fully connected classification head ensure feature extraction occurs primarily in the convolutional layers. Early stopping (15 epochs without validation F1 improvement) and learning rate reduction (factor 0.5 after two stagnant epochs) are used during training. All details are provided in Appendix A.3.

2.3.1 Baseline models

Baseline classification models are first trained to verify that each pre-processing pipeline allows viable classification. The baseline classification models all possess the same architecture. All baseline models use three convolutional layers with 32, 64, 128 filter numbers respectively, which led to networks with between around 38,000 and 45,000 parameters in each pipeline. A macro-F1 score of at least 0.7 on the test set is used as the success criterion.

2.3.2 Models for Experimentation

A grid search was conducted over CNN filter-number configurations to identify architectures of different sizes to investigate the effect of network capacity on modularity. The number of filters controls the width of each layer; this isolates capacity effects from confounds of receptive field size (i.e., kernel size) or hierarchical depth (i.e., layer number). The goal was to select one small, one medium, and one large CNN architecture to be used consistently across all three pipelines. All networks followed the same structure as the baseline classification model: three convolutional layers with kernel sizes of 7, 5, and 3 respectively.

Final candidate architectures were selected according to the following criteria:

1. Parameter count: small networks (approximately 500–1500 parameters), medium networks (approximately 3000–5000 parameters), and large networks (approximately 10,000–15,000 parameters)

2. Best validation accuracy within the first 10 training epochs must reach at least 0.7

3. Criteria 1 and 2 must be met across all three pipelines with identical filter counts in each layer

2.4 Differentiable Weight Masking

To investigate emergent modularity patterns across varying network capacities and input representations, the differentiable weight masking approach introduced by Csordás et al. (2020) is adapted and extended. The original methodology identifies subnetworks responsible for specific tasks or classes by identifying which weights are required in a network to perform a subset of the tasks the original network was trained on.

2.4.1 Mask Training Procedure

Sub-networks are identified by training masks that set weights in the original network to either 0 or 1 to identify the weights needed to fulfill a subset of the original network's tasks. The binary mask is applied to all weights of the convolutional layers via element-wise multiplication. Following Csordás et al. (2020), fully connected layer weights are excluded from masking to prevent classifier removal from artificially dropping target class performance to zero, which would confound analysis of performance in convolutional layers.

The modularity analysis follows these steps:

1. **Control Mask (M_{control}):** The binary weight mask obtained by training on the complete classification task (all C classes) to identify a correct loss-scaling factor α and as a reference for performance for the class-exclusion masks.

2. **Class-Exclusion Mask($M_{\mathbf{c}}$):** The binary weight mask obtained by training on data from all classes **except** a class c. The network's original weights remain frozen during this mask training. One class-exclusion mask is trained for every class included in the original classification task.

Mask training uses learning rate 0.01, Gumbel temperature τ=1.0, gradient clipping threshold 1.0, and initial mask logits of 2.0 (around 88% probability). α candidates are tested from 10^{-8} upward.

2.4.2 Loss Function and Regularization

Following Csordás et al. (2020), mask parameters are trained as logits and binarized via Gumbel-Sigmoid sampling with straight-through estimation. This allows the network to learn which weights to retain while maintaining differentiability. A temperature of $\tau = 1.0$ is used throughout. See Appendix A.4 for the full implementation details.

2.4.3 Methodological Adaptations

To address small-network constraints, several adaptations were made to the masking procedure from Csordás et al. (2020). First, mask accuracy is evaluated on the validation set at every epoch (rather than every fifth) to detect rapid degradation. Second, an adaptive α-reduction scheme is used: starting from a baseline α that maintains $\geq 95\%$ of unmasked control accuracy, α is halved whenever (a) accuracy falls below this threshold (reverting to the last valid state) or (b) no sparsity improvements occur for five consecutive epochs. This gradual reduction reflects that critical weights form a larger proportion of sparser networks, requiring gentler pruning pressure. Training stops after five reductions or 50 epochs. A complete description, including the rationale and validation checks, is provided in Appendix A.5.

2.5 Operational Definitions for Network Analysis

As discussed in Section 1.2, Csordás et al. (2020) did not apply their methodology to CNNs. Their sharing metric is ill-defined for the mask training regime they used for CNNs. Therefore, their framework is extended and further specified to allow the application to CNNs, provide clear definitions and operationalisation of terminology and test the authors' underlying assumptions. In the following M_{all} is refering to the mask in which all weights are set to active, which corresponds to the full network. Only the masks that were found during the mask differentiation will be abbreviated with M.

Based on the paper, several different types of weights are specified.

2.5.1 Excluded weights of a class c

The set of weights that are active in the full network but become inactive when class c is removed from training. The weights are not necessarily associated with the classification of class c itself. The set of weights that are active in the full network but are not in M_c due to the removal of class c from training:

$$E_c = M_{\text{all}} \setminus M_c$$

This study will explicitly differentiate between excluded weights and weights functionally relevant for a class.

2.5.2 Core Weights

The set of weights that remain active regardless of which single class is excluded during mask training. Formally, it is the set of weights in

the full mask that are *not* part of any exclusion set:

$$W_{\text{core}} = M_{\text{all}} \setminus \bigcup_{c=1}^{C} E_c$$

These are identified by removing the union $\bigcup$ of all classes' excluded weights from the full mask M_{all}. The performance drops observed by Csordás et al. (2020) following class-exclusion showed that CNNs rely on 'class-exclusive, non-shared features' and therefore concluded that CNNs 'fail to share weights'. They did not test whether the weights that remain shared across all classes are functionally competent. To directly test this, the core network's performance and distribution of predictions are evaluated.

2.5.3 Reconstructed Class Weights

A minimal subnetwork constructed by combining the core weights with the excluded weights for class c:

$$R_c = W_{\text{core}} \cup E_c$$

This subnetwork is created to test whether the excluded weights correspond to what Csordás et al. (2020) described as weights "solely responsible for individual classes and not shared by any other".
In the present paper, excluded weights will only be operationalized as specialized for a class c if their addition to the core network increases performance on class c compared to the core network alone. If adding these weights does not improve performance, they are likely redundant with respect to class c rather than functionally specialized.

2.5.4 Shared weights between classes c and d

Shared weights of classes c and d are weights that are active in the reconstructed class mask of both class c and d. Csordás et al. (2020) introduced shared weights as those that are active in two single-task masks, not the class-exclusion masks trained for CNNs. The reconstructed class weights are used to allow a calculation of shared weights within CNNs while using Csordás et al. (2020)'s masking approach. The shared weights are identified as:

$$\text{Shared}_{cd} = R_c \cap R_d$$

where $R_c \cap R_d$ denotes the intersection of active weights in both masks.

2.6 Derived Network Component Metrics

2.6.1 Exclusion Proportion

For a class c, the fraction of the full network hypothesized to be specialized for it (the size of its excluded set).

$$P^c_{\text{excl}} = \frac{|E_c|}{|M_{\text{all}}|}$$

2.6.2 Global Exclusion

The network-wide average is

$$P_{\text{excl}} = \frac{1}{C} \sum_{c=1}^{C} P^c_{\text{excl}}$$

2.6.3 Pairwise Sharing Proportion

The pairwise sharing metric was adapted from Csordás et al. (2020) by using reconstructed subnetworks instead of the single-task masks. Pairwise sharing denotes the proportion of class c's reconstructed subnetwork (R_c) that is shared with class d's subnetwork (R_d).

$$S_{cd} = \frac{|R_c \cap R_d|}{|R_c|}$$

where $|\cdot|$ returns the number of active weights and $R_c \cap R_d$ denotes the intersection of active weights in both masks. The number of intersection weights is scaled by the number of weights in R_c, making it a pairwise and asymmetric metric.

2.6.4 Global Pairwise Sharing

A generalized measure of the sharing metric is formulated for multi-class networks. For a network with C classes, let $S \in \mathbb{R}^{C \times C}$ be the weight sharing matrix where each element S_{cd} is calculated by using the reconstructed class masks of classes c and d instead of the original masks M_c and M_d. Global sharing is computed as:

$$S_{\text{global}} = \frac{1}{C(C-1)} \sum_{c=1}^{C} \sum_{d=1 d \neq c}^{C} S_{cd}$$

2.6.5 Summary

From the full network (M_{all}), three components are derived: excluded weights (E_c), core weights (W_{core}), and reconstructed weights

(R_c). Figure 4 visualizes these components and their relationships.

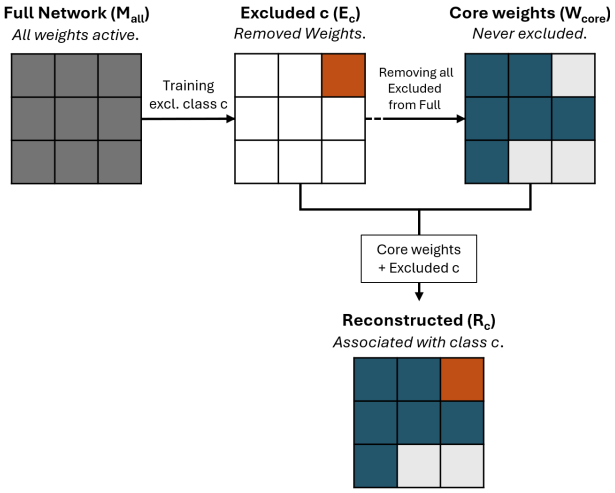


Figure 4: Visual summary of network components and their relationships. From the full network (M_{all}), excluded weights (E_c), core weights (W_{core}), and reconstructed weights (R_c) are derived. Arrows indicate derivation paths.

To address RQ2.1, the following metrics quantify modularity characteristics:

- **Exclusion proportion (P_{excl}).** The class-specific fraction of the network. A high proportion suggests specialization.
- **Core proportion (P_{core}).** The fraction of the network shared across *all* classes. A high proportion indicates a large common computation base.
- **Pairwise sharing (S_{cd}).** The overlap between reconstructed subnetworks for classes c and d. The global average (S_{global}) is reported across all class pairs. High sharing indicates that many weights are shared between classes.

The performance and prediction distribution of the core are evaluated to assess functional competence (RQ2.2). Above-chance performance would challenge Csordás et al. (2020)'s inference that performance drops after class exclusion imply an absence of sharing. Following Csordás et al. (2020)'s approach, the effect of removing excluded weights (E_c) on class c performance is tested (RQ2.3). Reconstructed weights (R_c) test whether excluded weights support class c independently of other classes' excluded weights (RQ2.3).

3 Results

3.1 Classification Models

3.1.1 Baseline Models

To establish that sleep stage classification is feasible under each pre-processing pipeline, baseline models were first trained and evaluated. Table 1 presents the accuracy and F1 scores for the baseline models across the three pre-processing pipelines. All pipelines successfully exceed the 70% F1 threshold.

Table 1: Baseline model performance across pre-processing pipelines

Pipeline	No. Param	No. Chann	Acc/F1
Baseline (32-64-128 filters)			
P1	37,925	10	0.83/0.77
P2	45,093	42	0.78/0.74
P3	40,165	20	0.78/0.72

Note: Acc/F1 = full network performance on test set.

No. Param = total trainable parameters in convolutional layers only. No. Chann = input channel count after pre-processing: P1 preserves raw channels (10), P2 extracts bandpower features (42), P3 applies PCA reduction (20 components). All models exceed 0.70 F1 threshold.

3.1.2 Selected Models

To investigate how network capacity interacts with pre-processing, three model sizes (small, medium, large) were selected per pipeline according to predefined criteria (Table 2). All met the 0.70 validation F1 criterion within 10 epochs, though final test performance varied. Small models (583-1,479 params) reached 0.76-0.80 accuracy. Medium models (2,957-4,749 params) matched or slightly exceeded small-model performance. Large models (10,261-13,845 params) showed greater pipeline-dependent variation: P1 peaked at an accuracy of 0.84 (F1=0.76), while P3 declined to an accuracy of 0.76 (F1=0.71). Notably, medium and large models outperformed the larger baseline architectures, suggesting baseline overfitting.

Table 2: Models selected for Analysis

Pipeline	No. Param	No. Chann	Acc/F1
Small capacity (4-6-7 filters)			
P1	583	10	0.80/0.70
P2	1,479	42	0.79/0.72
P3	863	20	0.76/0.68
Medium capacity (8-16-32 filters)			
P1	2,957	10	0.80/0.70
P2	4,749	42	0.80/0.75
P3	3,517	20	0.79/0.73
Large capacity (16-32-64 filters)			
P1	10,261	10	0.84/0.76
P2	13,845	42	0.81/0.76
P3	11,381	20	0.76/0.71

Note: Acc/F1 = full network performance on test set.

No. Param = total trainable parameters in convolutional layers only. No. Chann = input channel count after pre-processing

3.2 Differentiable Weight Masking

3.2.1 Modularity Across Conditions

To quantify how capacity and pre-processing affect modular organization, global weight sharing (S_{global}), core proportion (P_{core}), and exclusion proportion (P_{excl}) were computed for each condition and are presented in Table 3. In addition, the full-network performance is reported across all experimental conditions. Summary statistics that show the average and standard deviations of the modularity and performance metrices across capacities and pipelines can be found in Table 7 and 8 in the Appendix.

Table 3: Modularity metrics across network capacities and pre-processing pipelines

Pipeline	S_{global}	P_{core}	P_{excl}	Acc/F1
Small capacity (4-6-7 filters)				
P1	0.792	0.333	0.264	0.80 / 0.70
P2	0.835	0.497	0.193	0.79 / 0.72
P3	0.862	0.176	0.594	0.76 / 0.68
Medium capacity (8-16-32 filters)				
P1	0.851	0.246	0.469	0.80 / 0.70
P2	0.919	0.062	0.785	0.80 / 0.75
P3	0.918	0.069	0.789	0.79 / 0.73
Large capacity (16-32-64 filters)				
P1	0.863	0.509	0.226	0.84 / 0.76
P2	0.869	0.403	0.350	0.81 / 0.76
P3	0.846	0.403	0.126	0.76 / 0.71

Note: S_{global} = global weight sharing proportion, P_{core} = core weight proportion, P_{excl} = average exclusion proportion per class. Acc/F1 = full network test performance. All values are proportions (0-1). Darker cells indicate higher values within each metric column.

Pipeline trends Pre-processing systematically affected modular organization. Global sharing increased with disentanglement: P1 (M=0.835, SD=0.038), P2 (M=0.874, SD=0.042), and P3 (M=0.875, SD=0.038). Core proportion decreased as inputs became more disentangled, from P1 (M=0.363, SD=0.134) to P2 (M=0.216, SD=0.171) and P3 (M=0.216, SD=0.171). Conversely, exclusion increased: P1 (M=0.320, SD=0.131), P2 (M=0.503, SD=0.341), and P3 (M=0.503, SD=0.341).

Taken together, these trends indicate that input disentanglement promotes a specific organizational strategy: weights are reused across multiple classes (higher S_{global}), but few weights are shared across all classes (lower P_{core}). Instead, each class recruits a larger set of exclusion weights (E_c) not needed by others.

Performance patterns diverged as P1 achieved highest accuracy (M =0.813, SD=0.023) but intermediate F1 (M=0.720, SD=0.035); P2 yielded highest F1 (M=0.743, SD=0.021) with slightly lower accuracy (M=0.800, SD=0.010); P3 showed lowest performance (Acc M=0.770, SD=0.017; F1 M=0.707, SD=0.025).

Capacity trends Medium networks showed highest global sharing (M=0.896, SD=0.039), lowest core proportion (M=0.126, SD=0.104), and highest exclusion (M=0.681, SD=0.184), extensive pairwise sharing without a universal core. Large networks exhibited larger cores (M=0.438, SD=0.061) with lower exclusion (M=0.234, SD=0.112). Small networks fell between.

Performance increased modestly with capacity: small (Acc M=0.783, SD=0.021; F1 M=0.700, SD=0.020), medium (Acc M=0.797, SD=0.006; F1 M=0.727, SD=0.025), large (Acc M=0.803, SD=0.040; F1 M=0.743, SD=0.029). Performance variability was highest in large networks.

Summary These findings address RQ2.1. Organizational strategy varied with capacity: medium networks favored pairwise sharing over a universal core, while large networks developed both. Pipeline effects were inconsistent; disentanglement increased exclusion on average but diverged sharply at small capacity before converging at large capacity.

Crucially, global pairwise sharing exceeded the maximum FNN sharing reported by Csordás et al. (2020) (around 65%) in all nine conditions.

Table 4: Core network vs. full network performance across capacities and pipelines

Pipeline	Core Acc	Core F1	Full Acc	Full F1
Small capacity (4-6-7 filters)				
P1	0.51	0.33	0.80	0.70
P2	0.47	0.37	0.79	0.72
P3	0.22	0.17	0.76	0.68
Medium capacity (8-16-32 filters)				
P1	0.64	0.50	0.80	0.70
P2	0.08	0.03	0.80	0.75
P3	0.36	0.30	0.79	0.73
Large capacity (16-32-64 filters)				
P1	0.71	0.53	0.84	0.76
P2	0.75	0.68	0.81	0.76
P3	0.70	0.65	0.76	0.71

Note: Core values represent performance of W_{core} (weights shared across all classes). Full values show complete network performance for comparison. Darker cells indicate higher values within each metric column.

3.2.2 Core Network

The performance of the core networks (W_{core}) and the distribution of predictions were evaluated on the full test sets to assess functionality. Performance and the number of classes predicted by the core networks varied significantly.

Performance Core accuracy ranged from 0.08 (P2-medium) to 0.75 (P2-large); F1 from 0.03 to 0.68 (Table 4).

To determine whether core networks perform above chance, a control experiment was conducted with randomly permuted labels, yielding a chance accuracy of 0.229 (macro F1=0.075). Across all nine networks, core accuracy (M=0.493, SD=0.235) significantly exceeded chance (t(8)=3.38, p=0.0097; Wilcoxon p=0.014). Seven of nine networks performed above chance individually.

By capacity, large cores achieved consistently high accuracy (M=0.720, SD=0.026; all above chance). Medium cores showed high variability (M=0.360, SD=0.280; group n.s.), though two of three exceeded chance individually. Small cores (M=0.400, SD=0.157; group n.s.) also had two of three above chance.

By pipeline: P1 cores performed reliably above chance (M=0.620, SD=0.101; p=0.022). P2 and P3 cores showed greater variability and did not reach group significance, though individual exceptions existed (e.g., P2-large). Exploratory two-way ANOVAs found no significant main effects of capacity (p=0.10) or pipeline (p=0.58), consistent with non-monotonic interactions.

Table 5: Distribution of Core Predictions

	Wake	N1	N2	N3	REM
True Distribution					
	32.7 %	8.3 %	25.3 %	19.3 %	14.4 %
Small capacity					
P1	30.6 %	0.0 %	26.0 %	43.4 %	0.0 %
P2	33.0 %	7.2 %	2.7 %	50.1 %	6.9 %
P3	0.0 %	0.0 %	0.7 %	7.4 %	91.9 %
Medium capacity					
P1	46.9 %	14.9 %	16.9 %	21.2 %	0.0 %
P2	0.0 %	100.0 %	0.0 %	0.0 %	0.0 %
P3	0.0 %	39.4 %	46.5 %	14.0 %	0.0 %
Large capacity					
P1	49.1 %	1.8 %	27.0 %	21.4 %	0.8 %
P2	32.6 %	15.8 %	23.4 %	20.9 %	7.3 %
P3	30.7 %	13.9 %	18.8 %	17.0 %	19.6 %

Note: Values represent percentage of total predictions for each sleep stage class. True Distribution shows actual class prevalence in test data. All values are percentages, with each row summing to 100% (except rounding differences).

Distribution of Predictions Table 5 shows core prediction distributions across sleep stages. Most extreme: P3-small predicted REM 91.9% of the time; P2-medium predicted N1 100% of the time. Both have among the smallest cores (<0.18 proportion). Notably, REM and N1 are the least frequent classes in training, suggesting some cores implement a single "default" pathway rather than multi-class discrimination.

Small/medium cores tended toward imbalanced predictions (e.g., favoring N3 or REM). Large cores, especially P2/P3, produced more balanced distributions predicting all classes.

Core proportion alone does not determine prediction variance: P3-medium ($P_{core} < 0.07$) showed higher variance than P2-medium ($P_{core} = 0.06$).

Summary These findings answer RQ2.2., CNNs do develop functionally competent core networks, weights shared across all classes that can classify on their own. Seven of nine cores performed above chance, with large-network cores achieving mean accuracy of 0.720.

However, competence is not guaranteed. Cores that performed below chance (e.g., P2-medium: Acc=0.08, F1=0.03) predicted only a single class, suggesting they function as default pathways rather than genuine multi-class classifiers. The prediction distributions show that functional cores (like P2-large) distributed predictions across all five stages, while non-functional cores collapsed onto one or two classes.

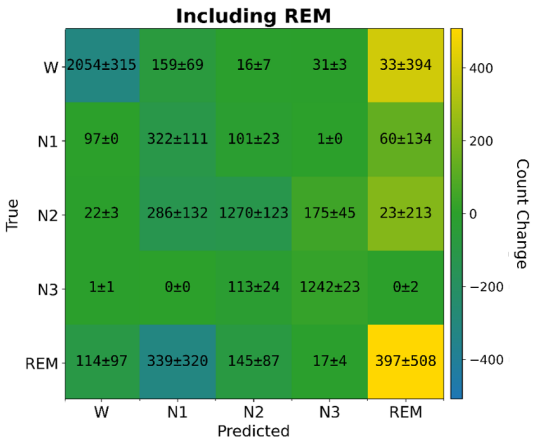
Crucially, the existence of any above-chance cores contradicts Csordás et al.'s inference that performance drops after class exclusion imply an absence of shared weights. The question is not whether sharing occurs, but under what conditions shared weights remain functional.

3.2.3 Effect of Exclusion and Reconstruction on Performance

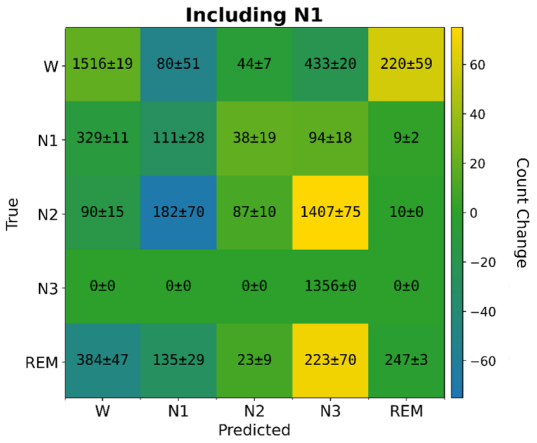
To assess whether class-excluded weights (E_c) function as dedicated class detectors, two analyses were performed: (1) exclusion: removing E_c from the full network, and (2) reconstruction: adding E_c to the core network ($W_{\text{core}} \cup E_c$). If these weights are specialized detectors, exclusion should harm target class performance and reconstruction should restore it.

Average Effects Performance effects of exclusion consistently exceeded those of inclusion, particularly in large networks. Exclusion caused substantial performance degradation on average, decreasing accuracy by 50.5 % on the target classes (Acc SD=0.285; Mean ΔF1=-0.465, SD=0.281), with strongest effects in medium-capacity networks and Pipeline 3. Reconstruction yielded more variable and weaker benefits on average, increasing accuracy by 36.3 % (Acc SD=0.364, Mean ΔF1=+0.169, SD=0.276).

Violations of the Detector Assumption The expected "exclusion-hurts, inclusion-helps" pattern was frequently violated. While exclusion decreased accuracy by up to -0.919, it improved target class accuracy in other cases (up to +0.051). Similarly, reconstruction degraded it in others (down to -0.218). Three violation patterns emerged: (1) both operations degrade performance, (2) exclusion helps while inclusion harms, (3) exclusion causes large drops but inclusion provides minimal benefit.



(a) Inclusion of REM weights primarily increases true REM predictions (Pipeline 2-large).



(b) Inclusion of N1 weights decreases N1 predictions while increasing others, suggesting a differentiation function (Pipeline 2-small).

Figure 5: Examples of specialized functional roles for excluded weights. (a) Class recognition: weights act as detectors for their target class. (b) Class differentiation: weights suppress confusion with the target class.

Functional Roles of Excluded Weights Analysis of confusion matrices shows that exclusion and reconstruction sometimes affected non-target classes more than the target class. Effects were sometimes asymmetric: harming one class did not correspond to helping that same class upon reconstruction. Four distinct functional patterns were observed:

- **Class Recognition**: Weights primarily increase correct predictions for their target class (e.g., REM weights in P2-large, Figure 5a.)
- **Feature Recognition**: Weights increase target class predictions across *multiple*

true classes, suggesting overgeneralized feature detection (e.g., N2 weights in P2-large).

- **Class Differentiation**: Weights decrease target class predictions while increasing others, suggesting they suppress confusion with similar stages (e.g., N1 weights in P2-small, Figure 5b).

- **Diffuse/Paradox**: Weights primarily affect non-target classes or increase false positives in a specific non-target class.

These patterns demonstrate that excluded weights serve diverse functional roles. While some approximate class detectors, others perform differentiation, feature generalisation, or distributed functions that are only coherent in combination with other weights. The class-exclusion mask procedure does not isolate dedicated detectors; it isolates weights that are *necessary* in context, which may serve inhibition, disambiguation, or feature sharing rather than class-specific recognition.

Summary These findings directly answer RQ2.3, which asked (a) whether class-excluded weights function as dedicated class detectors, and (b) whether adding them to the core selectively restores target class performance.

On (a): The weights do not reliably function as dedicated detectors. Exclusion and reconstruction often affected non-target classes more than the target classes. Four patterns emerged (recognition, feature detection, differentiation, and diffuse); only the first pattern fits the expected pattern. The others suggest more complex roles, including inhibition and disambiguation.

On (b): Partial, but inconsistent. Exclusion caused substantial degradation, confirming these weights are necessary in context. However, reconstruction benefits were weaker and more variable. Lastly, exclusion and reconstruction often showed asymmetric effects where exclusion harming one class did not correspond to helping that same class upon reconstruction. This challenges the assumption that excluded weights are solely "class detectors" (Csordás et al., 2020). The class-exclusion mask procedure does not isolate dedicated detectors; it isolates weights that are necessary in context with others.

4 Discussion

This study investigated how network capacity and input pre-processing shape emergent functional modularity in 1D CNNs trained on sleep stage classification. Following Csordás et al. (2020), modules were conceptualized as weights necessary for specific classes, identified through class-exclusion training, and their overlap and function were analyzed.

While prior work examined modularity under capacity constraints Csordás et al. (2020) and identified input disentanglement as a driver of specialization Béna and Goodman (2025) and Johnston and Fusi (2023), three gaps remained: weight sharing in CNNs had never been directly measured; the function of weights surviving all class exclusions remained untested; and no study had systematically manipulated capacity and input disentanglement within the same framework. The present study addressed these gaps by (a) introducing the core network construct, (b) adapting pairwise sharing metrics to CNNs, and (c) systematically varying capacity (small, medium, large) and pre-processing (P1, P2, P3) in a 3×3 design.

Weight Sharing in CNNs and Core performance Global pairwise sharing quantifys weight overlap between class-specific masks. In all conditions, the maximum of 0.65 Csordás et al. (2020) reported in FNNs was exceeded. In fact, it exceeded 0.80 in eight of nine conditions and 0.79 in the remaining condition (Table 3).

The proportion of core weights (shared across all classes) varied from 0.06 to 0.50. Core networks achieved above-chance performance in seven of nine cases. Even a core comprising only 7% of the network exceeded random performance, and all large-network cores exceeded 70% accuracy. Prediction distributions varied substantially: P2-medium's core predicted only N1, while P1-medium's core predicted four classes despite identical capacity. All large-network cores predicted all classes. This variance is not explained by core size alone as P3-medium ($P_{core} = 0.07$) showed more balanced predictions than P2-medium ($P_{core} = 0.06$). Networks appear to adopt multiple viable or-

ganizational strategies for identical tasks.

Csordás et al. (2020) reported that removing class-specific weights impaired performance on that class, concluding that CNNs "exhibit a consistent pattern of failing to share weights", the inference was drawn without quantitative overlap analysis. In the present study, core networks, with *no* class-excluded weights mostly performed above chance. Adhering to Csordás et al. (2020)'s conceptualization, such performance must indicate the presence of functionally competent shared weights. In the present study, an extension to pairwise sharing metric showed high proportions of sharing compared to the maximum reported by Csordás et al. (2020) in other networks. Their conclusion that CNNs resist sharing is therefore unsupported. The inference that the necessity of weights does inherently indicate the absence of sharing was refuted. This does not strictly indicate module reuse (their "P2"), as the methodology cannot identify concrete module functions. Differences in core network performance and prediction distributions indicate that CNNs adopt different internal organizations based on task and capacity.

Role of Excluded Weights The excluded weights were tested by assessing both the impact of their removal and the effect of adding them back to the core. The expectation that removal harms performance and addition restores it did not hold consistently.

Analysis of confusion matrices under weight addition revealed four patterns: (1) increased target-class predictions across all true classes, (2) increases primarily at true positives, (3) increased predictions of a different class that reduced target misclassifications, and (4) diffuse patterns.

The class-exclusion procedure thus isolates weights *necessary under competitive conditions* rather than weights solely responsible for one class Csordás et al. (2020). Some weights function as feature detectors rather than class detectors; others appear to differentiate by suppressing confusions with the target class. The assumption that excluded weights are "solely responsible for the target class" (Csordás et al., 2020) appears incomplete. The repeated observation of differentiation-like modules may point to functional building blocks that emerge within the task, similar to the concept of motifs.

Capacity and Input Pre-processing Interactions Medium-capacity networks developed the smallest cores (Mean $P_{\text{core}} = 0.126$, SD=0.104) yet the highest pairwise sharing (Mean $S_{\text{global}} = 0.896$, SD=0.039) and exclusion. Large networks developed both large cores and high sharing, suggesting sufficient capacity permits shared functional weights on which specialized weights are layered. Further processing (P2, P3) increased exclusion and pairwise sharing on average, but effects were highly capacity-dependent.

The relationship between capacity and modular organization is non-monotonic. These interactions were not predictable from prior work, which examined capacity and input structure in isolation. Deviations within the medium capacity condition suggest possible phase shifts due to the balance between task difficulty and capacity.

4.1 Limitations and Methodological Considerations

4.1.1 Class-Exclusion Mask Training

The masking procedure of Csordás et al. (2020) is unsuitable for imbalanced classes or differences in intrinsic separability.

Masks are trained to minimize cross-entropy on a class-excluded set. Because classes differ in prevalence and separability, the optimizer may minimize loss on easy classes at the expense of hard ones. Under sparsity pressure, the network could sacrifice one class's performance to improve another while increasing sparsity.

Between similar classes, removing one might improve performance on the other due to reduced confusion, making mask training less predictable. This may explain why exclusion sometimes *improved* target-class performance. Whether this reflects complex intra-module interactions (e.g., a module suppressing confusion might also suppress uncertain true positives) remains unclear.

What this does not affect:

- **Pairwise sharing (S_{global})**: Computed between masks trained identically; any bias affects both symmetrically.

- **Core performance (W_{core})**: Measured directly after all exclusions; independent of why weights were removed.
- **Heterogeneity of excluded weights**: Even if some weights were misidentified, the finding that CNNs has functionally diverse weights remains.

4.1.2 Scope and Generalizability

The experiments were limited to 1D CNNs, sleep staging, and three pre-processing pipelines. Whether these findings extend to 2D CNNs, other modalities, or non-medical domains remains open. However, sleep staging captures both distinct classes (Wake, REM) and classes on a continuum (N1–N3), structure common in hierarchical classification tasks (e.g., fine-grained categorization, bioacoustics, medical imaging). Consistency across three markedly different pipelines suggests the phenomena are not artifacts of particular feature representations.

This study did not test functional reuse across contexts (P2). The core demonstrates shared weights *can* classify, but not whether they are *reused* for distinct functions.

4.1.3 Statistical Power

Owing to computational constraints, each (capacity x pre-processing) condition was run with a single random seed. Run-to-run variability cannot be quantified. Multi-seed replication is necessary to confirm the stability of the results.

4.2 Implications and Future Research

4.2.1 Methodological Extensions

The limitations above point to concrete methodological improvements.

Loss function Focal loss or class-balanced variants during mask training would reduce the incentive to abandon difficult or rare classes, isolating function more cleanly from statistical convenience.

Mask parameterization. Gumbel-Sigmoid with global logit decay exerts constant off-pressure, which can over-prune small networks or miss critical weights. A more elegant alternative parameterizes masks as continuous values trained with a sigmoid, applying L1 regularization directly (Lepori et al., 2023). This encourages competitive sparsity as weights compete to stay on.

Future research should further develop Lepori et al. (2023)'s method: (1) Force binary weights in forward passes during later training stages, preventing dependence on non-binary weights. (2) Add intermittent validation checks. While Lepori et al. (2023) conducted more rigorous hyperparameter search than Csordás et al. (2020), final performance verification would confirm mask validity.

4.2.2 Architectural Research

Networks developed qualitatively different internal organizations, some cores predicted single minority classes, others distributed predictions broadly. This challenges interpretations of ablation experiments.

In literature, ablation is frequently combined with retraining after component removal to assess its effect (e.g., Meyes et al. (2019)). This procedure assumes a single optimal functional blueprint that retraining will recover minus the removed component.

Differences in core functionalities challenge this assumption in CNNs. If internal functional organization is not fixed, retraining may yield a new organization, meaning results reflect the new blueprint's competency, not the removed component's contribution. This aligns with findings that retrained networks underperform scratch-trained networks at equivalent sparsity (Díaz-Faloh & Mulet, 2025).

4.2.3 Signal Processing Research

A key observation is that CNNs spontaneously develop weights that differentiate between classes (e.g., suppressing N3 when N2 is present, or reducing false REM predictions during Wake). Most designed modular solutions lack such functionality. Attention mechanisms offer limited mitigation, as they require ample data (Hendricks et al., 2021).

Findings suggest research could: (1) explicitly train differentiation modules and integrate them in modular architectures (2) induce biases encouraging them for difficult classes, e.g., a general feature extraction block densely connected to distinct modules that connect only to some output nodes.

Most modular networks in physiological classification assign dedicated modules to channels (Zhu et al., 2020). Given the differentiation-like patterns observed, this approach may miss co-occurring cross-modal patterns.

4.3 Conclusion

This study provides a direct measurement of weight sharing in CNNs under the differentiable masking framework introduced by Csordás et al. (2020). Contrary to prior claims that CNNs "fail to share weights", global pairwise sharing exceeded 80% in nearly all conditions. The introduction of the *core network* construct demonstrated that weights shared across all classes are not merely present but are functionally competent, achieving above-chance performance in seven of nine networks and exceeding 70% accuracy in all large networks. Class-excluded weights were found to serve heterogeneous roles , such as: detection, feature generalisation, and differentiation. The class-exclusion procedure thus isolates weights necessary under competition rather than pure class encoders.

The effects of capacity and pre-processing were investigated; they interacted non-monotonically, indicating that modular organization does not scale linearly with resources and that effects of pre-processing are capacity-dependent.

These findings challenge the inference that performance drops following class exclusion imply an absence of sharing. The observation that CNNs spontaneously develop differentiation ability suggests that the mechanism might be a promising architectural prior that requires further investigation. The question is no longer *whether* CNNs share weights, but *how* they are shared.

References

Alcalá-Corona, S. A., Sandoval-Motta, S., Espinal-Enríquez, J., & Hernández-Lemus, E. (2021). Modularity in biological networks. *Frontiers in Genetics*, *12*, 701331. https://doi.org/10.3389/fgene.2021.701331

Amer, M., & Maul, T. (2019). A review of modularization techniques in artificial neural networks. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.1904.12770

Anderson, M. L. (2010). Neural reuse: A fundamental organizational principle of the brain. *Behavioral and Brain Sciences*, *33*(4), 245–266. https://doi.org/10.1017/s0140525x10000853

Béna, G., & Goodman, D. F. M. (2025). Dynamics of specialization in neural modules under resource constraints. *Nature Communications*, *16*(1), 187. https://doi.org/10.1038/s41467-024-55188-9

Bereska, L., & Gavves, E. (2024). Mechanistic Interpretability for AI Safety – A review. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.2404.14082

Clune, J., Mouret, J.-B., & Lipson, H. (2013). The evolutionary origins of modularity. *Proceedings of the Royal Society B Biological Sciences*, *280*(1755), 20122863. https://doi.org/10.1098/rspb.2012.2863

Csordás, R., Sjoerd, V. S., & Schmidhuber, J. (2020). Are neural nets modular? Inspecting functional modularity through differentiable weight masks. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.2010.02066

Díaz-Faloh, C., & Mulet, R. (2025). Diluting restricted Boltzmann machines. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.2511.00648

Filan, D., Casper, S., Hod, S., Wild, C., Critch, A., & Russell, S. (2021). Clusterability in neural networks. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.2103.03386

Hendricks, L. A., Mellor, J., Schneider, R., Alayrac, J.-B., & Nematzadeh, A. (2021). Decoupling the role of data, attention, and losses in multimodal transformers. *Transactions of the Association for Computational Linguistics*, *9*, 570–585. https://doi.org/10.1162/tacl\{_}a\{_}00385

Johnston, W. J., & Fusi, S. (2023). Abstract representations emerge naturally in neural networks trained to perform multiple tasks. *Nature Communications*, *14*(1), 1040. https://doi.org/10.1038/s41467-023-36583-0

Khalighi, S., Sousa, T., Santos, J. M., & Nunes, U. (2015). ISRUC-Sleep: A comprehensive public dataset for sleep researchers. *Computer Methods and Programs in Biomedicine*, *124*, 180–192. https://doi.org/10.1016/j.cmpb.2015.10.013

Lambert, I., & Peter-Derex, L. (2023). Spotlight on sleep stage classification based on EEG. *Nature and Science of Sleep*, *Volume 15*, 479–490. https://doi.org/10.2147/nss.s401270

Lee, Y. J., Lee, J. Y., Cho, J. H., & Choi, J. H. (2021). Interrater reliability of sleep stage scoring: a meta-analysis. *Journal of Clinical Sleep Medicine*, *18*(1), 193–202. https://doi.org/10.5664/jcsm.9538

Lepori, M. A., Serre, T., & Pavlick, E. (2023). Break it Down: Evidence for structural compositionality in neural networks. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.2301.10884

Li, K., Rubungo, A. N., Lei, X., Persaud, D., Choudhary, K., DeCost, B., Dieng, A. B., & Hattrick-Simpers, J. (2025). Probing out-of-distribution generalization in machine learning for materials. *Communications Materials*, *6*(1). https://doi.org/10.1038/s43246-024-00731-w

Liu, Y., Palacio, M.-I., Bikki, T., Toledo, C., Ouyang, Y., Li, Z., Wang, Z., Toledo, F., Zeng, H., & Herrero, M.-T. (2025). Machine Learning, Physiological Signals, and Emotional Stress/Anxiety: Pitfalls and challenges. *Applied Sciences*, *15*(21), 11777. https://doi.org/10.3390/app152111777

Liu, Z., Gan, E., & Tegmark, M. (2023). Seeing is believing: Brain-Inspired Modular Training for Mechanistic Interpretability. *Entropy*, *26*(1), 41. https://doi.org/10.3390/e26010041

Lorenz, D. M., Jeng, A., & Deem, M. W. (2011). The emergence of modularity in biolog-

ical systems. *Physics of Life Reviews*, *8*(2), 129–60. https://doi.org/10.1016/j.plrev.2011.02.003

Lu, Y., Yang, W., Zhang, Y., Chen, Z., Chen, J., Xuan, Q., Wang, Z., & Yang, X. (2021). Understanding the dynamics of DNNs using graph modularity. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.2111.12485

Malhotra, R. K., & Avidan, A. Y. (2013, October). *Sleep stages and scoring technique*. https://doi.org/10.1016/b978-1-4557-1267-0.00003-5

Meyes, R., Lu, M., Waubert, D. P. C., & Meisen, T. (2019). Ablation studies in artificial neural networks. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.1901.08644

Michoel, T., Joshi, A., Nachtergaele, B., & Van De Peer, Y. (2011). Enrichment and aggregation of topological motifs are independent organizational principles of integrated interaction networks. *Molecular BioSystems*, *7*(10), 2769–2778. https://doi.org/10.1039/c1mb05241a

Pfeiffer, J., Ruder, S., Vulić, I., & Ponti, E. M. (2023). Modular Deep learning. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.2302.11529

Praczyk, T. (2023). Emerging modularity during the evolution of neural networks. *Journal of Artificial Intelligence and Soft Computing Research*, *13*(2), 107–126. https://doi.org/10.2478/jaiscr-2023-0010

Qiu, Z., Huang, Z., & Fu, J. (2023). Unlocking emergent modularity in large language models. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.2310.10908

Ruis, L., & Lake, B. (2022). Improving systematic generalization through modularity and augmentation. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.2202.10745

Saganowski, S., Bródka, P., Koziarski, M., & Kazienko, P. (2019). Analysis of group evolution prediction in complex networks. *PLoS ONE*, *14*(10), e0224194. https://doi.org/10.1371/journal.pone.0224194

Woidtke, R. (2024). Sleep and the glymphatic system. *American nurse journal*, *19*(3), 06–11. https://doi.org/10.51256/anj032406

Zhu, T., Luo, W., & Yu, F. (2020). Multi-Branch Convolutional Neural Network for Automatic Sleep Stage Classification with Embedded Stage Refinement and Residual Attention Channel Fusion. *Sensors*, *20*(22), 6592. https://doi.org/10.3390/s20226592

A Study Appendix

A.1 Conceptualizations of Emergent Modularity in Literature

Table 6: Conceptualizations of Emergent Modularity in Neural Networks

Topological: Structural Modules are clusters with dense internal connections and sparse external connections (Filan et al., 2021).
Evolutionary: Structural, Functional, Induction Modules are connectivity clusters with sparse external connections and identifiable functionality. The focus is on evolutionary origin and necessity due to connection-cost (Clune et al., 2013; Praczyk, 2023).
Motif-based: Accumulation of small functional structures Modules are larger, self-contained structures that form due to the aggregation of motifs (small, recurring connection patterns). Those motifs perform small and specific information-processing functions (e.g., logical AND or OR) (Michoel et al., 2011).
Functional Modules are defined solely around functional specialization, regardless of structural organization.
Representational: Clusters in Network Activation Modules are defined based on activation-space geometry. Modules are formed by neurons or samples that cluster based on activation similarity (Lu et al., 2021; Qiu et al., 2023).

Some conceptualizations overlap with other constructs in research. Circuits, for example, are subgraphs responsible for specific behaviors (Bereska & Gavves, 2024). Those are not typically termed modules despite the alignment with motif-based or functional views of modularity.

A.2 Detailed Data and Pre-processing Specifications

Data The first subgroup of the ISRUC-Sleep dataset (Khalighi et al., 2015) is used. It contains overnight polysomnography (PSG) recordings from adult subjects suspected of sleep disorders collected at the Sleep Medicine Centre of the Hospital of Coimbra University. It contains one session of 100 participants.
PSG recordings were acquired at 200 Hz using the 10-20 electrode placement system. Each recording contains approximately 8 hours of overnight sleep data. The dataset includes 6 EEG channels (F3, C3, O1, F4, C4, O2), 2 EOG channels (LOC & ROC), 1 EMG channel of the chin and 1 ECG channel. Each session is split into 30-second epochs that were independently scored by two sleep experts according to AASM standards, with stages: Wake, N1, N2, N3, and REM sleep. The overall expert agreement rate exceeds 83% (Khalighi et al., 2015).
Out of the 100 participants 58 were included in the final classifications. 35 were excluded because of data issues (i.e., missing channels, or formatting differences) (IDs: 8, 14, 17, 23, 25, 30, 31, 32, 33, 34, 37, 38, 40, 41, 43, 48, 53, 54, 55, 58, 63, 65, 72, 75, 78, 80, 82, 83, 85, 86, 87, 88, 91, 92, 97). Additionally, the inter-rater agreement per participant epochs was used as a proxy for data quality. 7 participants whose overall interrater agreement rate was below 75% were excluded (IDs: 9, 11, 59, 60, 68, 71, 77).

Pre-processing Three different pre-processing pipelines are employed. This is done because previous research indicated that the dimensionality and disentanglement of the input can lead to specialisation of emerging modules. In addition, Johnston and Fusi (2024) found that low dimensional and disentangled data can lead to the development of specific sub-populations of neurons that fire exclusively in one context. In theory, a combination of generalised and specific modules would enable robust and efficient classification. Therefore, different pre-processing pipelines will be tested that produce different levels of disentanglement. After application of each of the pipelines, the 30 second epochs are extracted as arrays and saved as separate files. The number of channels

differs between the pipelines.

Pipeline One

- Pipeline one uses the least amount of processing. The goal is to clean the data but leave it entangled and correlated across the channels. A 50 Hz notch filter is applied to all channels to remove power line interference (Cooray et al., 2019). However, the goal of pipeline One is to lightly filter the raw signals to relevant frequencies but to still produce rich features that will be disentangled further during Pipeline Two and Three. Therefore, filtering is treated more liberally compared to classical sleep studies

- EEG: The EEG channels were band-pass filtered with 0.3-35 Hz to remove low frequency drift and slow artifacts and reduce high frequency noise and environmental interference (Cooray et al., 2019). This preserves the delta, theta, alpha and beta frequency components which are commonly used to detect sleep related brain rhythms.

- EOG: The band pass filter ranges recommended for EOG signals vary widely across research, a review by Boukadoum Ketones (1986) identified band pass ranges between 0.3 and 70 Hz. The authors recommend an upper cutoff between 8 Hz and 20 Hz. Therefore, a band pass filter of 0.1-15 Hz is applied to capture slow eye movement signals and provide a wider range for frequency band extraction in Pipeline 2.

- EMG: The low pass of EMG filters is recommended to be 400-500 Hz (Van Boxtel, 2001), however, due to the 200 Hz sampling rate of the signal the maximum possible frequency is 99 Hz. The recommendations of high pass filters are 15-25 Hz (Van Boxtel, 2001). However, Ferris et al. (2008) found that a submentalis muscle can be captured with 10-100 Hz bandpass filtering. Therefore, the chin EMG is processed with a band-pass filter ranging from 10-99 Hz.
  While Ferris et al. (2008) segmented the signal into 1-second epochs and calculated averages for each epoch, this method would create a 1 Hz signal which is too limiting as Pipelines 2 and 3 aim to further disentangle signals using the output of pipeline one.
  Therefore, it was decided to instead extract an envelope of the EMG signal after rectification, by low pass filtering the signal using a 4 Hz threshold. The high 4 Hz envelope was chosen to enable more extensive frequency band separation in Pipeline 2.

- ECG: A band pass filter between 0.5 - 45 Hz is applied to highlight the QRS complex (Oladunni & Aneni, 2025).

Pipeline Two
Pipeline two uses a moderate amount of processing. The uses the cleaned data from pipeline one and disentangled further by time varying bandpower features. Therefore, a ten second sliding window with stepsize of one second is used to calculate power spectral density, bandpower and subject-wise normalization by total power. The bandpower values are interpolated to the original time points to create a time varying signal similar to the one used resulting from pipeline one. Important to note is that all extracted bands use the entire range remaining after pipeline one, to ensure that every pipeline receives a processed version of the same data. This provides data conditions that are comparable across pipelines.

- EEG: Five frequency bands are extracted from all six EEG channels: Delta (0.3-4 Hz), Theta (4-8 Hz), Alpha (8-12 Hz), Sigma (12-16 Hz), Beta (15-35 Hz).

- EOG: Two bands are extracted to capture slow and fast eye movement separately: Slow (0.1-1 Hz) and medium (1-5 Hz), fast (5-15).

- EMG: Three bands were used to characterize muscle activity Low (5-15 Hz), Mid (15-30 Hz), High (30-99 Hz). (Mistake as it is a 4 Hz envelope).

- ECG: From Pipeline One: Three frequency bands are extracted from the ECG: Low (0.5-5 Hz), Medium (5-15 Hz), High (15-45 Hz). While 15-45 Hz are usually excluded from the analysis, the band is retained for data consistency, it might capture noise or higher harmonics of the QRS.

Pipeline Three

Pipeline Three receives the processed signal of pipeline two as input. The pipeline first incrementally fits a StandardScaler to normalize features, then incrementally fits the PCA model. Components are selected to collectively explain 95% of total variance. Component's loading pattern is analyzed to identify which original bandpower features (e.g., frontal delta, occipital alpha) contribute most strongly. Components are whitened to ensure equal variance across all dimensions. PCA is fitted exclusively on training and validation splits to prevent test data leakage.

When processing test epochs, the pipeline uses the pre-fitted PCA and scaler from the training data.

Test data undergoes the exact same transformation pipeline (scaling -> PCA projection) but without influencing the component directions or scaling parameters. The scaler calculates the mean and standard deviation for each of the 40 features across all training epochs. The PCA identified the principal directions (components) in the training data that capture the most variance. These are fixed vectors in the 40-dimensional feature space. Each scaled test sample is projected onto these pre-defined component directions.

Training, Test and Validation Data and Split Expert annotations, demographic information and sleep statistics were extracted and used to create one information file for each epoch of each subject.

Metadata about participant demographics (e.g., age, sex) and sleep statistics (e.g., total time sleep, sleep efficiency) were extracted from one of the experts' rating files. The interrater agreements in ratings were analysed. Their overall agreement per participant was calculated and added for later use as a proxy for data quality. Both expert ratings were added to the information file, with an annotation that indicates their agreement/disagreement on the epoch.

Only epochs that received the same stage rating by both experts and whose participant wide inter-rater agreement exceeded 75% were used for training, testing and validation of the models. This led to the inclusion of 58 participants and 43476 total epochs, with 11874 Wake, 3139 N1, 13064 N2, 8760 N3, and 6639 REM epochs. A participant wise training (70%), testing (15%) and validation (15%) split was employed to prevent data leakage. Additionally, each split was created using stratified sampling to achieve a similar sleep stage distribution in every split compared to the global distribution (limitation: only of those that the experts agreed upon). The final training, validation and test set contain 30833, 5625, 7018 samples. The same split is used for all models to increase comparability and reduce computational load.

A.3 Detailed Model Architectures

This study uses 1D CNNs to investigate how input representation and network size influence emergence of modularity. Two model families were utilized: (1) baseline models to verify the viability of the pre-processing pipelines with a standardized architecture, and (2) models with varied capacity to investigate network size effects on modularity. All CNNs used in this study share the following characteristics:

- Hierarchical feature extraction: Three convolutional layers with decreasing kernel sizes (7, 5, 3) are supposed to provide sufficient depth for hierarchical feature learning while remaining interpretable. Decreasing kernel sizes are used to capture coarse to fine temporal patterns.
- Global Average Pooling (GAP) bottleneck: Following the final convolutional layer, GAP compresses spatial information into a fixed-length vector, creating an information bottleneck that has been shown to encourage filter specialization (Csordás et al., 2020).
- All models use ReLU activations, max pooling between layers, and a single fully connected classification head. This ensures that feature extraction occurs primarily in the convolutional layers.
- During training early stopping and a learning rate reduction are implemented. Early stopping triggers after 15 epochs without improvement on the validation F1-score. The learning rate starts with 1.00e-03 and is reduced by a factor of 0.5 after two epochs without validation loss improvements. The maximum epoch number is 50 and the batch size is 32.

A.4 Detailed Loss Function and Regularization

The mask parameters are represented as logits, to which Gumbel noise is added before applying a sigmoid function, producing probabilities between 0 and 1. During training, these probabilities are sampled to obtain binary values using a straight-through estimator that thresholds at 0.5 while maintaining gradient flow. These binary masks are applied element-wise to the frozen network weights, and the resulting masked network is evaluated using cross-entropy loss. The total loss is calculated as the general cross entropy loss summed with an L1 regularization term scaled by α and the batch size:

$$L_{\text{total}} = L_{\text{task}} + \alpha \cdot B \sum_{i=1}^{N} l_i$$

Where:

- L_{task} is the cross-entropy loss
- α is the regularization strength
- B is the batch size
- l_i are the mask logits
- N is the total number of convolutional weights

The batch size scaling is used to scale the regularization strength by batch size so that α represents the regularization strength per sample. The regularization term $\alpha B \sum_i l_i$ penalizes mask logits rather than probabilities, encouraging sparse mask distributions.

A.5 Detailed Methodological Adaptations for Mask Training

Diverging from Csordás et al. (2020), several methodological adaptations were made to address small-network constraints:

1. Epoch-wise accuracy monitoring: Mask accuracy is evaluated on the validation set at every epoch (rather than every fifth epoch) to detect rapid accuracy degradation.

2. Adaptive reduction of baseline α: First, Csordás et al. (2020)'s method is followed to determine an α-value that maintains $\geq 95\%$ of the unmasked models' accuracy in the control mask setting. The baseline value serves as initial value for all masks but is adapted when:
a) Accuracy relative to control mask performance falls below a 95% threshold, then training reverts to the last state satisfying the threshold and continues with α halved.
b) No sparsity improvements occur for five consecutive epochs, then α is halved to encourage further weight removal. This adaptation reflects that as the network becomes sparser, the proportion of critical weights increases. High α-values encourage aggressive pruning, which would hinder identification of smaller sets of to be pruned weights in later training stages.

Training stops after five α-reductions or after a maximum of 50 training epochs. These modifications were made to address the small size of the networks used, since failures to detect redundant weights would distort modularity measurements disproportionally compared to larger networks.

A.6 Full Network: Performance and Modularity Metrics

Table 7: Summary statistics of modularity metrics across capacities and pipelines

	S_{global}	P_{core}	P_{excl}
By Capacity			
Small	0.830 (0.035)	0.335 (0.161)	0.350 (0.214)
Medium	0.896 (0.039)	0.126 (0.104)	0.681 (0.184)
Large	0.859 (0.012)	0.438 (0.061)	0.234 (0.112)
By Pipeline			
P1	0.835 (0.038)	0.363 (0.134)	0.320 (0.131)
P2	0.874 (0.042)	0.321 (0.229)	0.443 (0.307)
P3	0.875 (0.038)	0.216 (0.171)	0.503 (0.341)
Overall			
All	0.862 (0.039)	0.300 (0.171)	0.422 (0.252)

Note: S_{global} = global weight sharing, P_{core} = core proportion, P_{excl} = average exclusion proportion. Values shown as mean (standard deviation) calculated across 3 networks per category.

Table 8: Summary statistics of classification performance across capacities and pipelines

	Accuracy	F1-score
By Capacity		
Small	0.783 (0.021)	0.700 (0.020)
Medium	0.797 (0.006)	0.727 (0.025)
Large	0.803 (0.040)	0.743 (0.029)
By Pipeline		
P1	0.813 (0.023)	0.720 (0.035)
P2	0.800 (0.010)	0.743 (0.021)
P3	0.770 (0.017)	0.707 (0.025)
Overall		
All	0.794 (0.025)	0.723 (0.029)

Note: Values shown as mean (standard deviation) calculated across 3 networks per category.

A.7 Core Network: Performance

A.7.1 Summary Statistics

Table 9: Summary statistics of the core classification performance across capacities and pipelines

	Accuracy	F1-score
By Capacity		
Small	0.407 (0.162)	0.290 (0.106)
Medium	0.360 (0.280)	0.277 (0.236)
Large	0.720 (0.026)	0.620 (0.079)
By Pipeline		
P1	0.620 (0.101)	0.453 (0.108)
P2	0.440 (0.338)	0.360 (0.325)
P3	0.427 (0.247)	0.373 (0.248)
Overall		
All	0.496 (0.235)	0.396 (0.216)

Note: Values shown as mean (standard deviation) calculated across 3 networks per category.

A.7.2 Accuracy and F1 in Detail

Table 10: Accuracy of the Core weights on the full dataset

	Wake	N1	N2	N3	REM	Overall
Small capacity						
P1	0.79	0.00	0.23	0.99	0.00	0.51
P2	0.66	0.19	0.05	1.00	0.24	0.47
P3	0.00	0.00	0.02	0.38	1.00	0.22
Medium capacity						
P1	0.95	0.37	0.51	0.88	0.00	0.64
P2	0.00	1.00	0.00	0.00	0.00	0.08
P3	0.00	0.83	0.65	0.66	0.00	0.36
Large capacity						
P1	0.90	0.10	0.76	0.93	0.05	0.71
P2	0.90	0.55	0.72	0.92	0.39	0.75
P3	0.84	0.46	0.54	0.81	0.68	0.70

Table 11: F1 of the Core weights on the full dataset

	Wake	N1	N2	N3	REM	Overall
Small capacity						
P1	0.82	0.00	0.22	0.61	0.00	0.33
P2	0.66	0.20	0.09	0.56	0.33	0.37
P3	0.00	0.00	0.04	0.55	0.27	0.17
Medium capacity						
P1	0.78	0.27	0.61	0.84	0.00	0.50
P2	0.00	0.15	0.00	0.00	0.00	0.03
P3	0.00	0.29	0.46	0.77	0.00	0.30
Large capacity						
P1	0.78	0.16	0.73	0.89	0.10	0.53
P2	0.90	0.38	0.74	0.88	0.52	0.68
P3	0.87	0.34	0.62	0.86	0.58	0.65

A.8 Reconstruction and Exclusion: Performance changes

A.8.1 Summary Statistics

Table 12: Exclusion performance changes across capacities and pipelines

	Δ Accuracy	Δ F1-score
By Capacity		
Small	-0.450 (0.355)	-0.452 (0.307)
Medium	-0.662 (0.230)	-0.637 (0.251)
Large	-0.404 (0.192)	-0.306 (0.177)
By Pipeline		
P1	-0.415 (0.303)	-0.357 (0.274)
P2	-0.532 (0.294)	-0.505 (0.275)
P3	-0.568 (0.254)	-0.534 (0.279)
Overall		
All	-0.505 (0.285)	-0.465 (0.281)

Note: Values represent mean (standard deviation) of performance drop when class-specific weights are excluded from the full network. Negative values indicate performance degradation. Larger absolute values indicate stronger specialization effects.

Table 13: Inclusion performance changes across capacities and pipelines

	Δ Accuracy	Δ F1-score
By Capacity		
Small	+0.307 (0.343)	+0.221 (0.249)
Medium	+0.499 (0.450)	+0.245 (0.343)
Large	+0.283 (0.258)	+0.042 (0.186)
By Pipeline		
P1	+0.267 (0.403)	+0.068 (0.231)
P2	+0.410 (0.383)	+0.216 (0.305)
P3	+0.412 (0.305)	+0.224 (0.278)
Overall		
All	+0.363 (0.364)	+0.169 (0.276)

Note: Values represent mean (standard deviation) of performance gain when class-specific weights are included in the core network. Positive values indicate performance improvement when weights are restored.

A.8.2 Corresponding Class

Table 14: Pipeline 1: Performance Differences When a Class is Excluded (vs. Full Model) and Included (vs. Core Model)

	Δ Acc Excl	Δ Acc Incl	Δ F1 Excl	Δ F1 Incl
Small capacity (4-6-7 filters)				
W	-0.022	+0.028	0.000	+0.003
N1	-0.129	0.000	-0.198	0.000
N2	-0.495	+0.059	-0.437	+0.092
N3	-0.701	+0.009	-0.580	-0.155
REM	-0.851	+0.929	-0.784	+0.455
Medium capacity (8-16-32 filters)				
W	-0.560	+0.055	-0.374	-0.193
N1	-0.129	+0.046	-0.190	-0.097
N2	-0.793	+0.392	-0.756	-0.062
N3	-0.311	-0.218	-0.177	-0.060
REM	-0.805	+1.000	-0.759	+0.283
Large capacity (16-32-64 filters)				
W	-0.024	+0.005	-0.039	-0.032
N1	-0.243	+0.642	-0.337	+0.166
N2	-0.258	+0.135	-0.128	-0.013
N3	-0.180	+0.010	-0.054	-0.012
REM	-0.725	+0.917	-0.537	+0.644

Note: **Exclusion** values show performance drop when class-specific weights are removed from the full network (Full Model - Excluded Model). **Inclusion** values show performance gain when class-specific weights are added to the core network (Included Model - Core Model). All values are differences (higher absolute value = more impact). Positive inclusion values indicate weights help that class; negative inclusion values indicate weights hurt that class. Colors: dark green = large positive, light green = small positive, light red = small negative, dark red = large negative.

Table 15: Pipeline 2: Performance Differences When a Class is Excluded (vs. Full Model) and Included (vs. Core Model)

	Δ Acc Excl	Δ Acc Incl	Δ F1 Excl	Δ F1 Incl
Small capacity (4-6-7 filters)				
W	-0.284	+0.238	-0.267	+0.225
N1	-0.003	-0.048	-0.013	-0.022
N2	-0.739	+0.443	-0.639	+0.411
N3	+0.051	-0.009	-0.191	+0.245
REM	-0.698	+0.537	-0.650	+0.103
Medium capacity (8-16-32 filters)				
W	-0.866	+0.928	-0.891	+0.883
N1	-0.456	-0.069	-0.424	+0.081
N2	-0.805	+1.000	-0.787	+0.434
N3	-0.897	+0.836	-0.888	+0.766
REM	-0.721	+0.993	-0.745	+0.320
Large capacity (16-32-64 filters)				
W	-0.348	+0.088	-0.209	-0.073
N1	-0.496	+0.410	-0.451	-0.084
N2	-0.740	+0.224	-0.669	-0.091
N3	-0.409	+0.080	-0.252	-0.087
REM	-0.576	+0.502	-0.494	+0.131

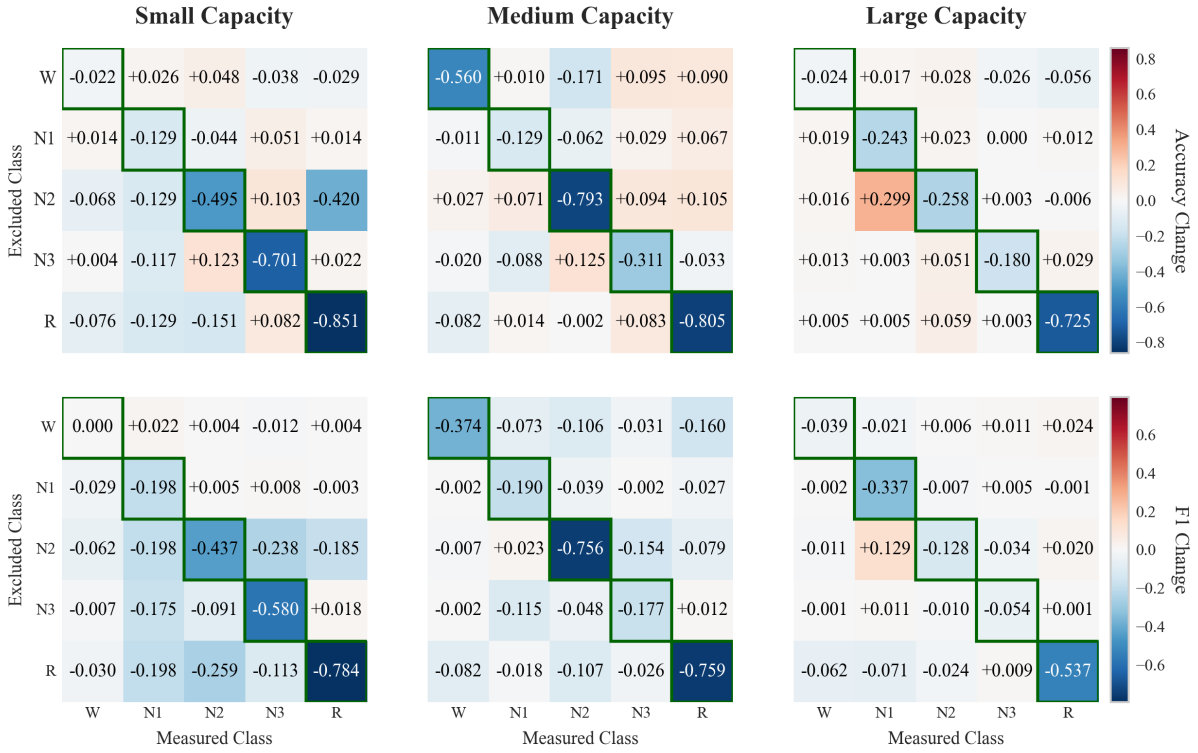
Note: **Exclusion** values show performance drop when class-specific weights are removed from the full network (Full Model - Excluded Model). **Inclusion** values show performance gain when class-specific weights are added to the core network (Included Model - Core Model). All values are differences (higher absolute value = more impact). Positive inclusion values indicate weights help that class; negative inclusion values indicate weights hurt that class. Colors: dark green = large positive, light green = small positive, light red = small negative, dark red = large negative.

Table 16: Pipeline 3: Performance Differences When a Class is Excluded (vs. Full Model) and Included (vs. Core Model)

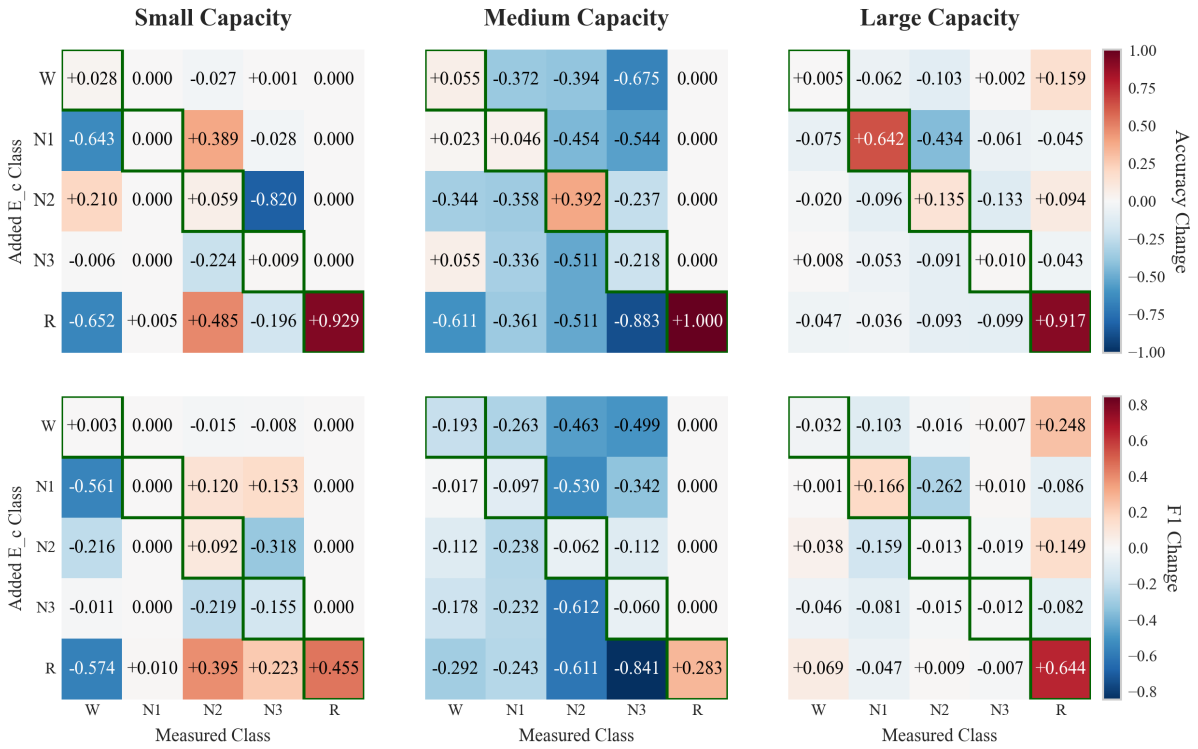
	Δ Acc Excl	Δ Acc Incl	Δ F1 Excl	Δ F1 Incl
Small capacity (4-6-7 filters)				
W	-0.889	+0.921	-0.875	+0.821
N1	-0.256	+0.401	-0.300	+0.220
N2	-0.812	+0.624	-0.772	+0.471
N3	-0.822	+0.502	-0.884	+0.337
REM	-0.096	-0.025	-0.197	+0.106
Medium capacity (8-16-32 filters)				
W	-0.919	+0.993	-0.900	+0.735
N1	-0.515	+0.148	-0.435	+0.010
N2	-0.753	+0.338	-0.687	+0.063
N3	-0.812	+0.247	-0.867	+0.063
REM	-0.585	+0.798	-0.674	+0.445
Large capacity (16-32-64 filters)				
W	-0.472	+0.137	-0.308	-0.055
N1	-0.456	+0.422	-0.369	-0.005
N2	-0.421	+0.345	-0.245	+0.128
N3	-0.317	+0.108	-0.208	-0.004
REM	-0.390	+0.223	-0.290	+0.021

Note: **Exclusion** values show performance drop when class-specific weights are removed from the full network (Full Model - Excluded Model). **Inclusion** values show performance gain when class-specific weights are added to the core network (Included Model - Core Model). All values are differences (higher absolute value = more impact). Positive inclusion values indicate weights help that class; negative inclusion values indicate weights hurt that class. Colors: dark green = large positive, light green = small positive, light red = small negative, dark red = large negative.

A.8.3 Across Classes

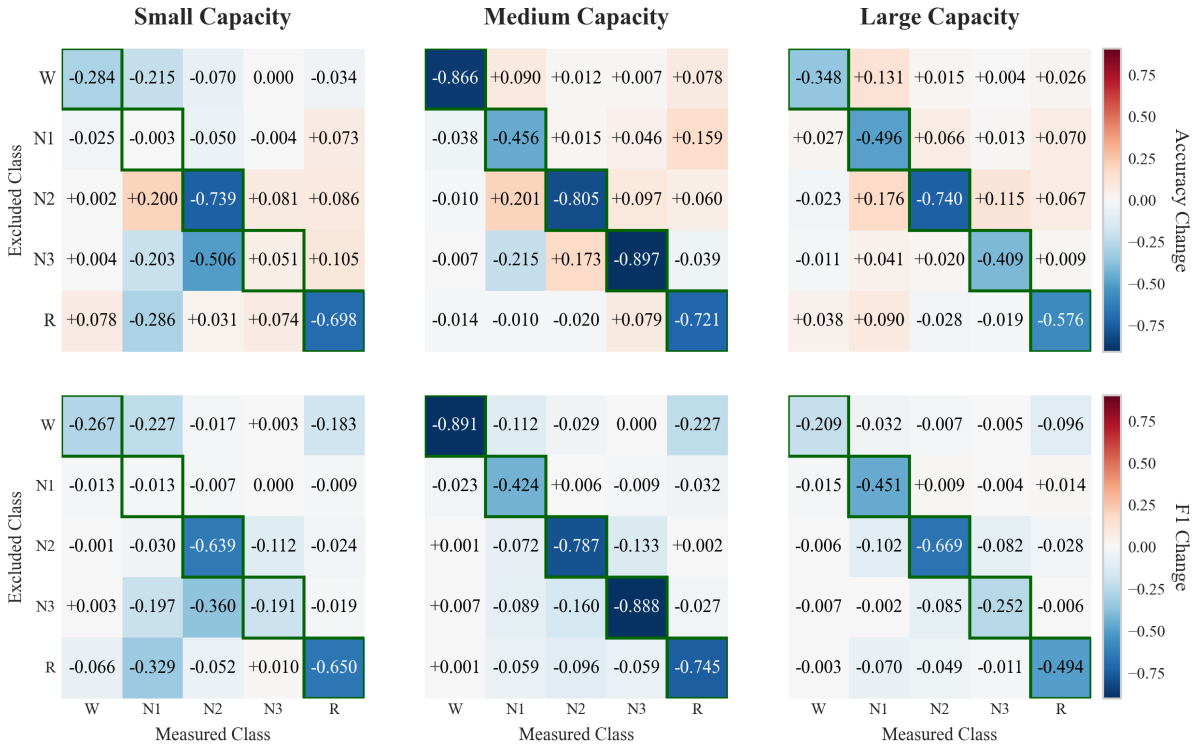


(a) Exclusion analysis: Mask_excl - Baseline

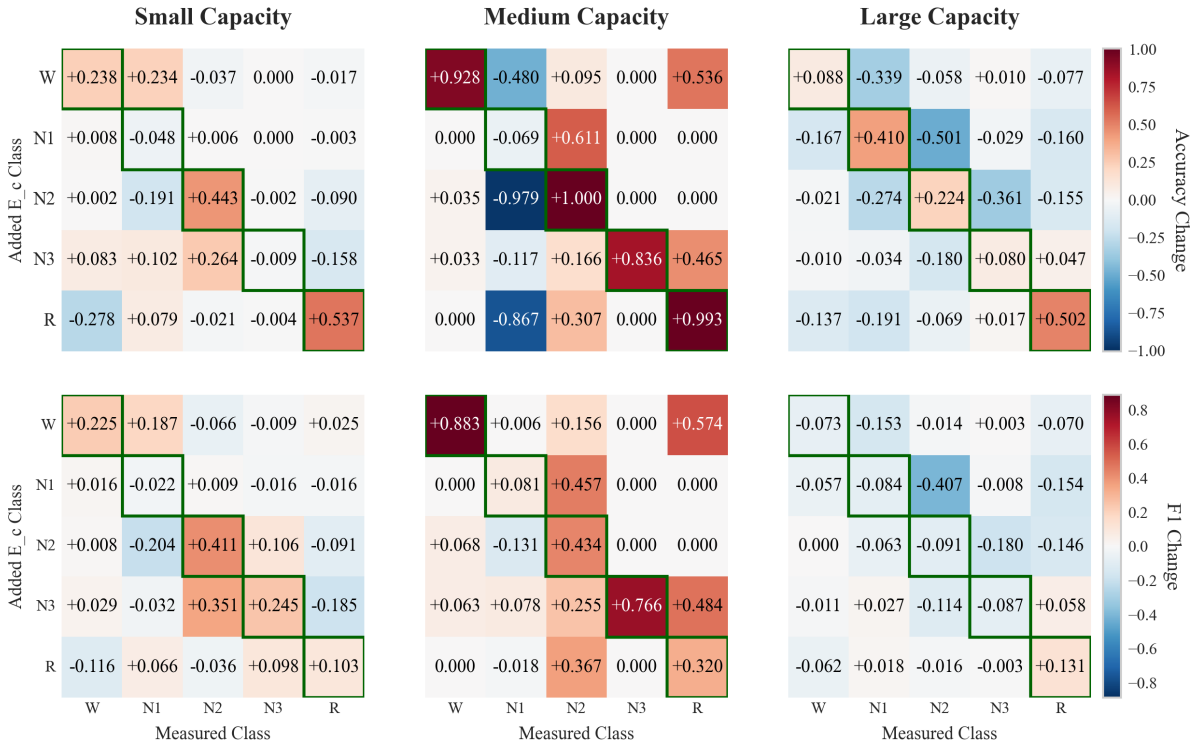


(b) Reconstruction analysis: R_c - Core_only

(c) Pipeline 1 analysis comparison: (a) Exclusion analysis showing performance change when removing class-specific weights, and (b) Reconstruction analysis showing performance change when adding class-specific weights. Both analyses use identical visualization parameters for direct comparison.

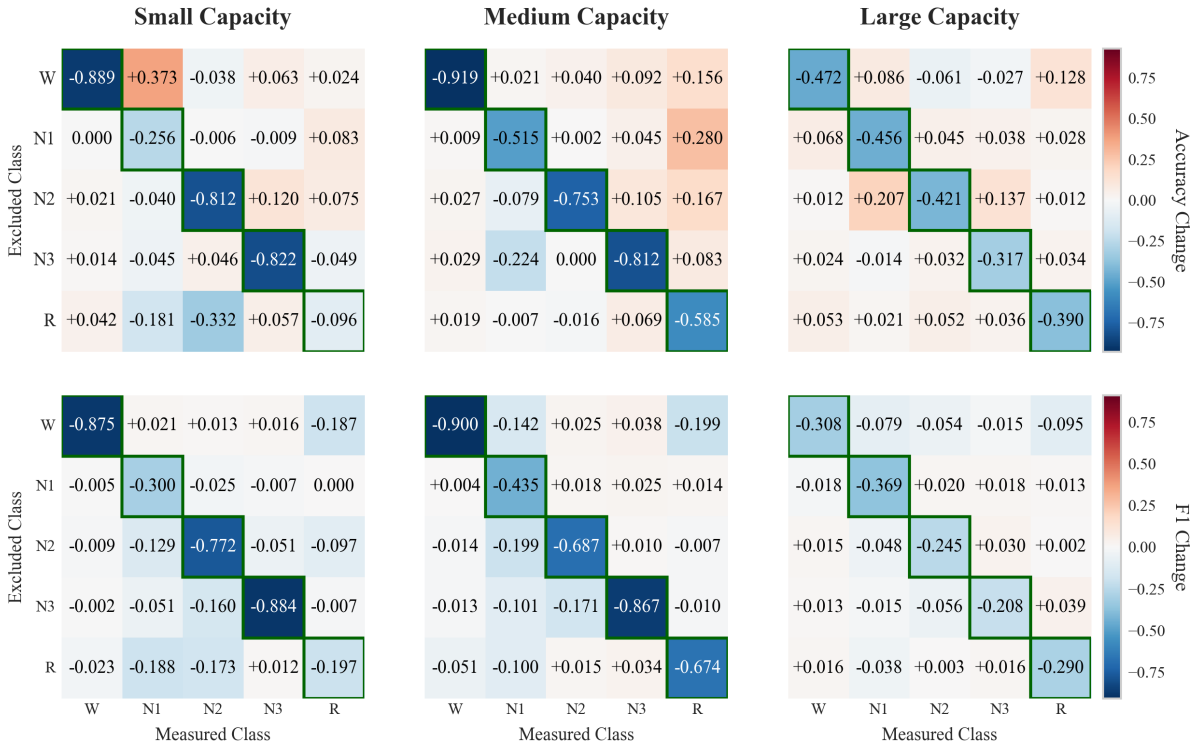


(a) Exclusion analysis: Mask_excl - Baseline



(b) Reconstruction analysis: R_c - Core_only

(c) Pipeline 2 analysis comparison: (a) Exclusion analysis showing performance change when removing class-specific weights, and (b) Reconstruction analysis showing performance change when adding class-specific weights. Both analyses use identical visualization parameters for direct comparison.



(a) Exclusion analysis: Mask_excl - Baseline



(b) Reconstruction analysis: R_c - Core_only

(c) Pipeline 3 analysis comparison: (a) Exclusion analysis showing performance change when removing class-specific weights, and (b) Reconstruction analysis showing performance change when adding class-specific weights. Both analyses use identical visualization parameters for direct comparison.

A.8.4 Confusion Matrices: Class-Exclusion

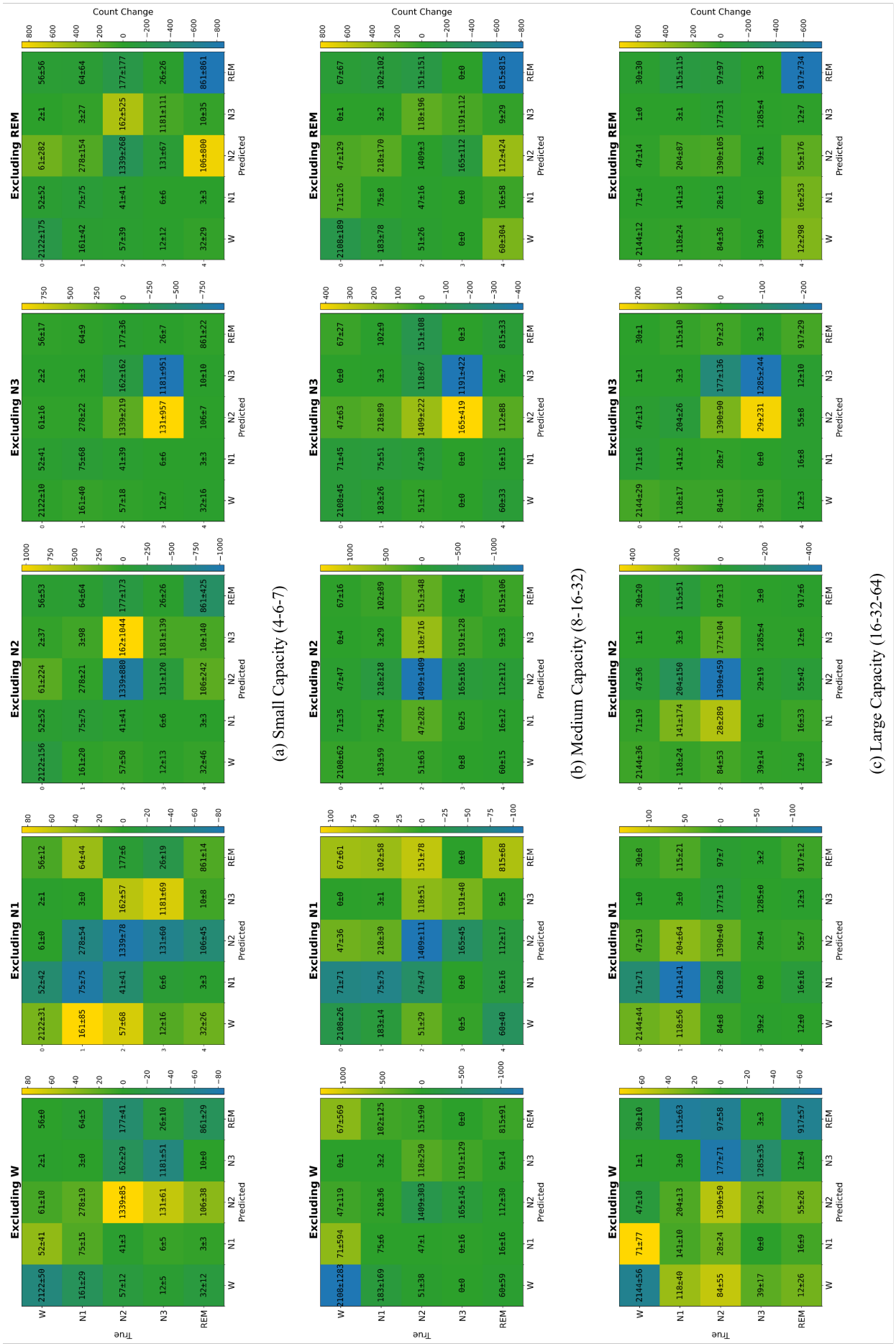
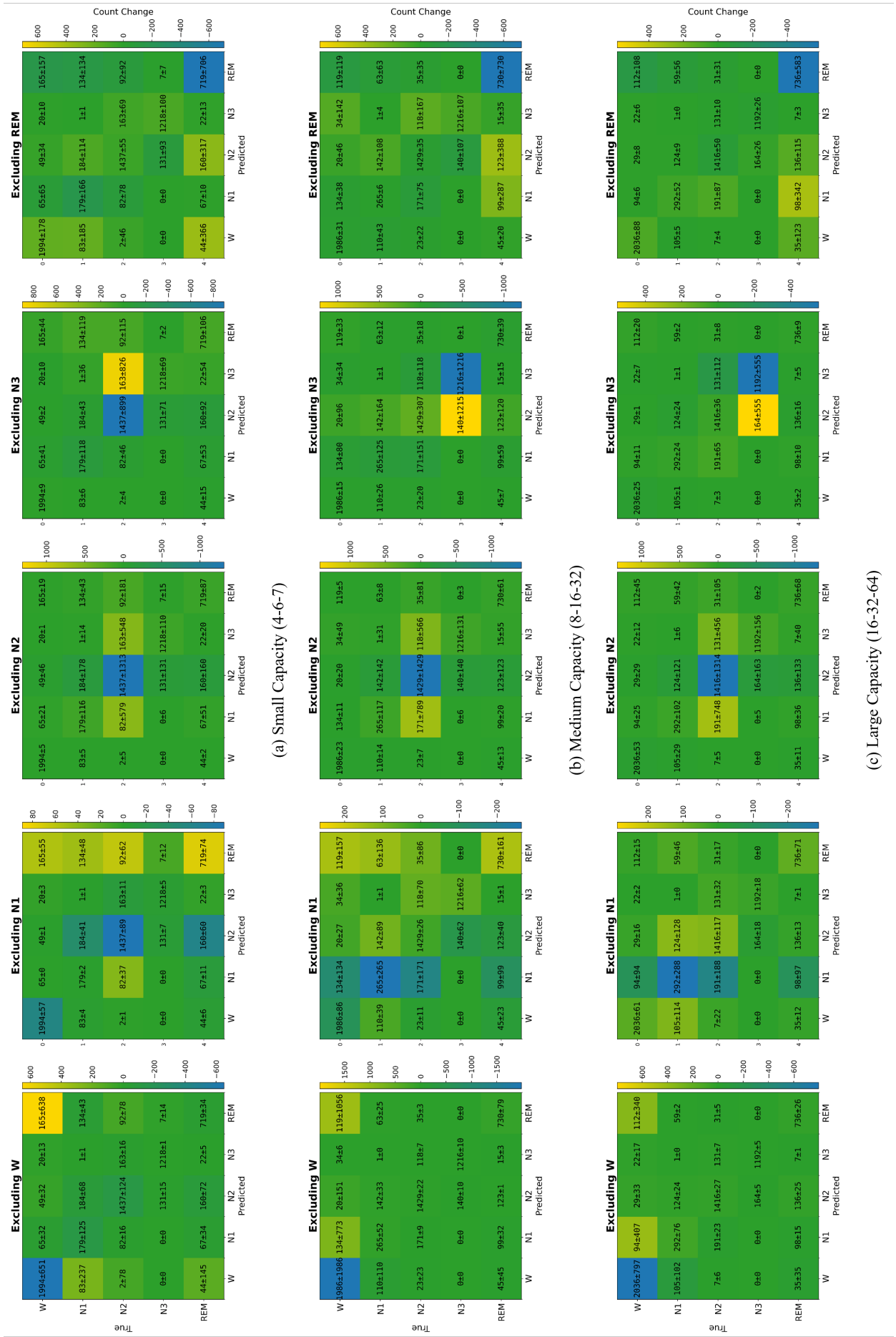


Figure 9: Pipeline One: True-label confusion matrix differences for class-exclusion experiments. Each cell displays baseline count from the full network ± change induced by applying the Exclusion Mask.

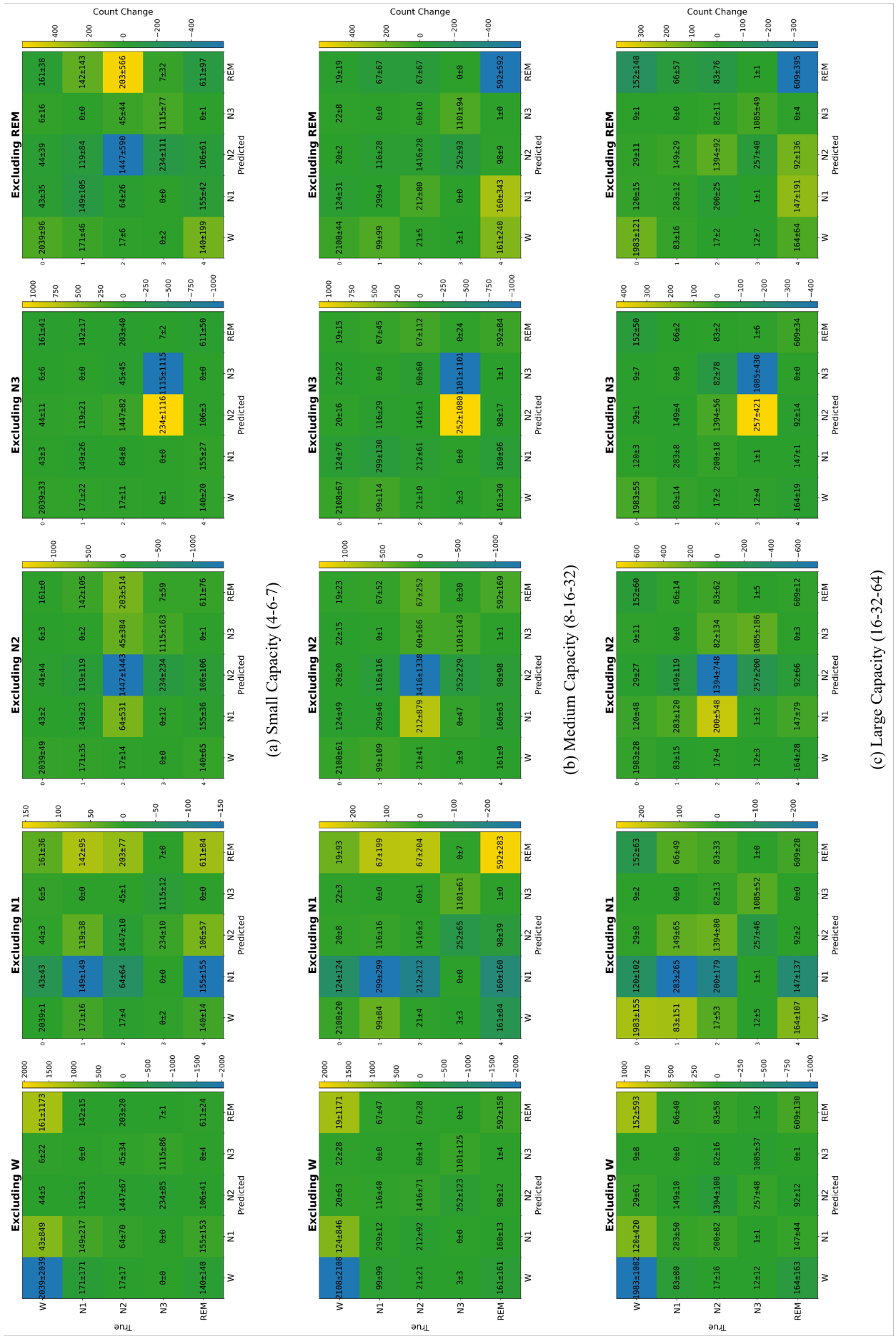


(a) Small Capacity (4-6-7)

(b) Medium Capacity (8-16-32)

(c) Large Capacity (16-32-64)

Figure 10: Pipeline Two: True-label confusion matrix differences for class-exclusion experiments. Each cell displays baseline count from the full network ± change induced by applying the Exclusion Mask.



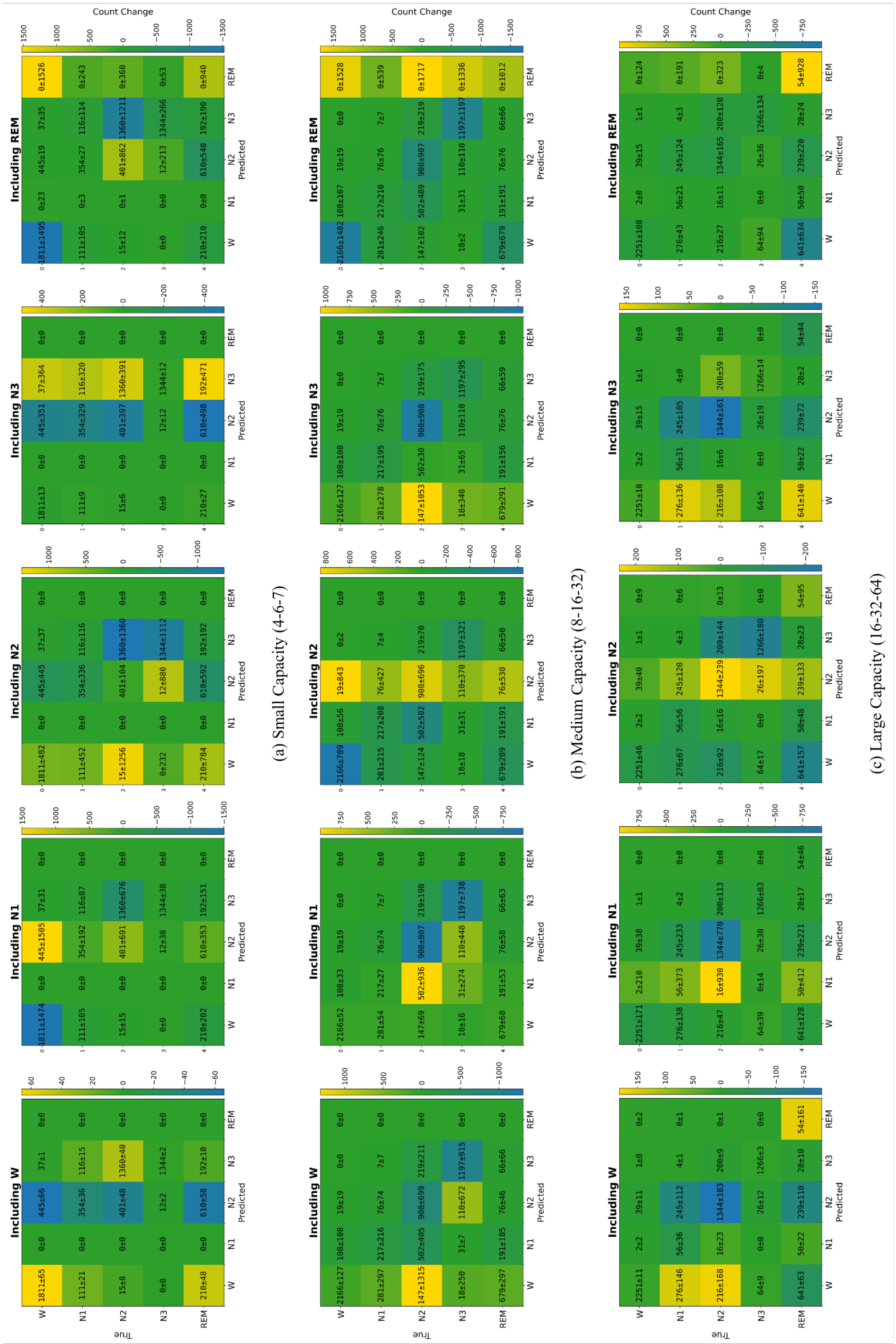
(a) Small Capacity (4-6-7)

(b) Medium Capacity (8-16-32)

(c) Large Capacity (16-32-64)

Figure 11: Pipeline Three: True-label confusion matrix differences for class-exclusion experiments. Each cell displays baseline count from the full network ± change induced by applying the Exclusion Mask.

A.8.5 Confusion Matrices: Class-Inclusion

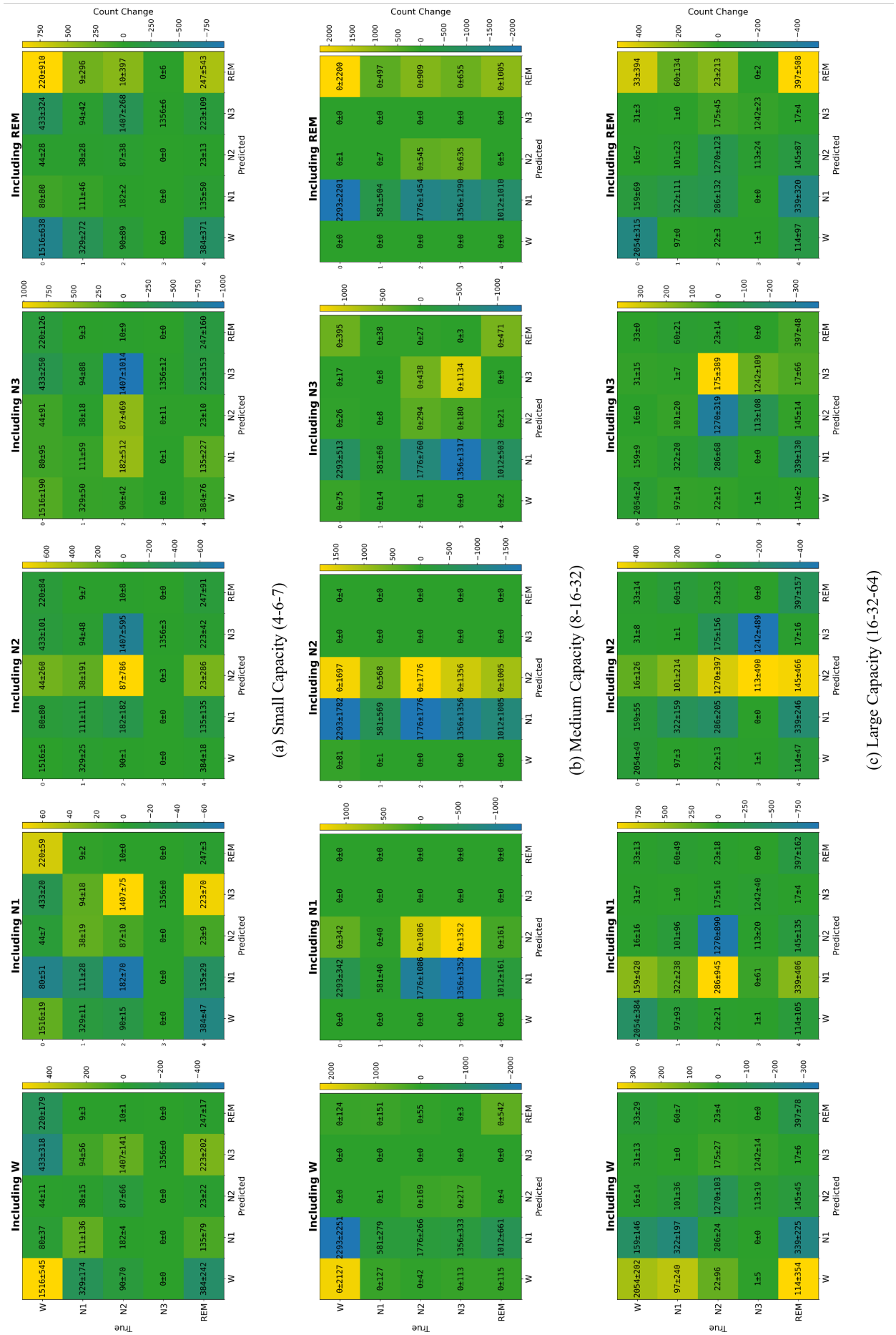


(a) Small Capacity (4-6-7)

(b) Medium Capacity (8-16-32)

(c) Large Capacity (16-32-64)

Figure 12: Pipeline One: True-label confusion matrix differences when comparing Core Network to a Reconstructed Network containing core and the excluded weights of a class. Each cell displays baseline count from the Core network ± change induced by adding the Class weights.

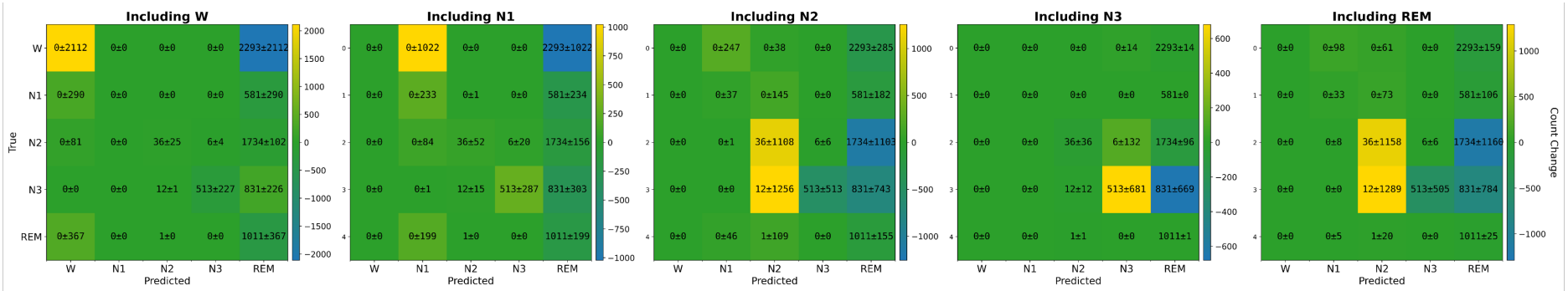


(a) Small Capacity (4-6-7)

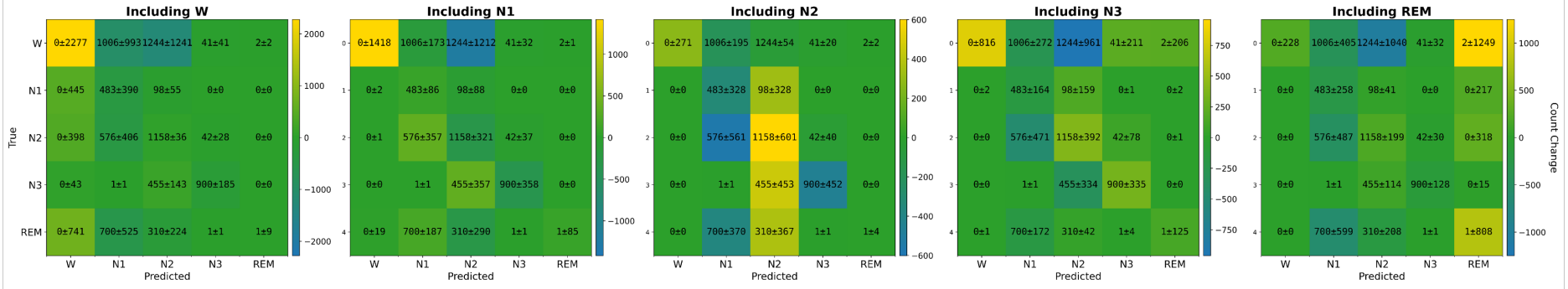
(b) Medium Capacity (8-16-32)

(c) Large Capacity (16-32-64)

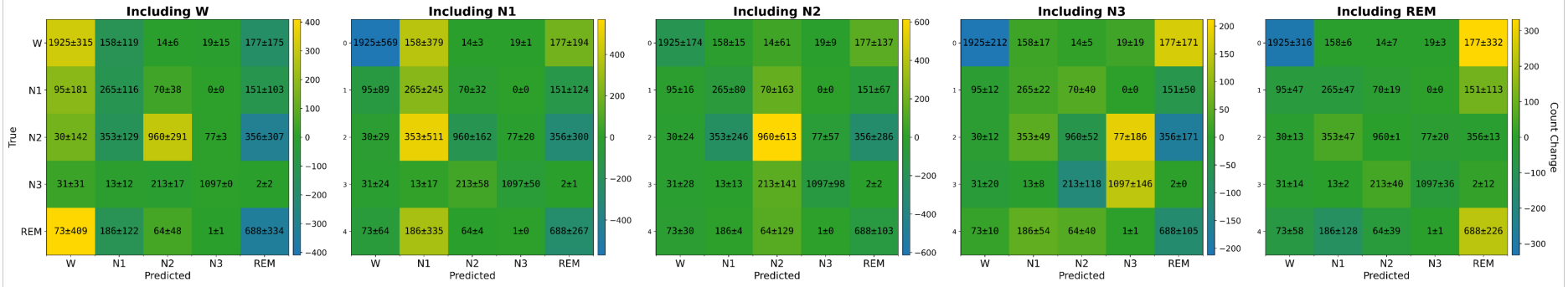
Figure 13: Pipeline Two: True-label confusion matrix differences when comparing Core Network to a Reconstructed Network containing core and the excluded weights of a class. Each cell displays baseline count from the Core network ± change induced by adding the Class weights.



(a) Small Capacity (4-6-7)



(b) Medium Capacity (8-16-32)



(c) Large Capacity (16-32-64)

Figure 14: Pipeline Three: True-label confusion matrix differences when comparing Core Network to a Reconstructed Network containing core and the excluded weights of a class. Each cell displays baseline count from the Core network ± change induced by adding the Class weights.